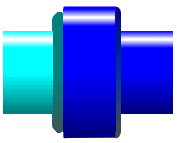
ALPHA MICROSYSTEMS
RIGHT. FROM THE START.

AlphaDDE Programmer's Reference Guide



DSM-00208-00 Rev A00

© 1995 Alpha Microsystems

REVISIONS INCORPORATED	
REVISION	DATE
Preliminary	December 1994
A00	May 1995

AlphaDDE Programmers Reference Guide

To re-order this document, request part number DSM-00208-00 Rev A00

The information contained in this manual is believed to be accurate and reliable. However, no responsibility for the accuracy, completeness or use of this information is assumed by Alpha Microsystems.

This document was written by John Riblett and Peter Winter.

Illustrations by Peter Winter.

This document may contain references to products covered under U.S. Patent Number 4,530,048.

The following are registered trademarks of Alpha Microsystems, Santa Ana, CA 92799:

AMIGOS	AMOS	Alpha Micro	AlphaACCOUNTING
AlphaBASIC	AlphaCALC	AlphaCOBOL	AlphaDDE
AlphaFORTRAN 77	AlphaLAN	AlphaLEDGER	AlphaMAIL
AlphaMATE	AlphaNET	AlphaPASCAL	AlphaRJE
AlphaWRITE	CASELODE	OmniBASIC	VER-A-TEL
VIDEOTRAX			

The following are trademarks of Alpha Microsystems, Santa Ana, CA 92799:

AlphaBASIC PLUS	AlphaVUE	AM-PC	AMTEC
DART	ESP	MULTI	*in*Sight/am
*in*Front/am			

All other copyrights and trademarks are the property of their respective holders.

ALPHA MICROSYSTEMS
2722 S. Fairview St.
P.O. Box 25059
Santa Ana, CA 92799

Table of Contents

TABLE OF CONTENTS ..... **i**

CHAPTER ONE INTRODUCTION **1**

COMPONENTS OF ALPHADDE 1

CHAPTER TWO INSTALLATION AND CONFIGURATION NOTES **3**

REQUIREMENTS FOR ALPHADDE 3

INSTALLATION 3

INITIAL TEST 4

CONFIGURATION NOTES 4

CHAPTER THREE GENERAL PHILOSOPHY **5**

DDE CONCEPTS AND IMPLEMENTATION 5

ALPHADDE MESSAGE FLOW 5

DEVELOPER REQUIREMENTS 6

A NOTE ON NAMES 6

TOPIC SERVERS 6

FUNCTIONAL DESCRIPTION OF TOPIC SERVERS 6

CHAPTER FOUR FUNCTIONAL GROUPS **9**

SETUP 9

SIMPLE MESSAGING 9

MESSAGING FUNCTIONS 10

HOT (ADVISE) LINK SUPPORT 10

CHAPTER FIVE ALPHADDE FUNCTION REFERENCE SHEETS **13**

dde'add'hotlnk (basic) *14*

dde'advise'all (basic) *15*

dde'connect (basic) *16*

dde'delete'hotlnk (basic) *18*

dde'disconnect (basic) *19*

dde'get'request (basic) *20*

dde'process'request (basic) *22*

dde'return'string & dde'return'unf (basic) *24*

dde'wait'request (basic) *25*

DDECON.SBR (xcall) *27*

DDEDIS.SBR (xcall) *28*

DDEGET.SBR (xcall) *29*

DDEGTR.SBR (xcall) *31*

DDEPRC.SBR (xcall) *33*

DDESNA.SBR (xcall) *35*

DDESNR.SBR (xcall) *36*
DDESVA.SBR (xcall) *38*
DDECON (assembler) *40*
DDEDIS (assembler) *42*
DDEGET (assembler) *43*
DDEGTR (assembler) *45*
DDEPRC (assembler) *47*
DDESNA (assembler) *49*
DDESNR (assembler) *50*
DDESVA (assembler) *52*

CHAPTER SIX PROGRAMMING AND INTEGRATION METHODS 55

M68 ASSEMBLY LANGUAGE TECHNIQUES 55

CHAPTER SEVEN SAMPLE TOPIC SERVERS AND WINDOWS DDE PROGRAMS 57

SAMPLE PROGRAMS 57
SAMPLE AMOS TOPIC SERVERS (IN DDE:, [7,55]) 57
SAMPLE PC CODE 57

CHAPTER EIGHT USING DDE IN WINDOWS APPLICATIONS 59

SAMPLE APPLICATION USAGE 59
LOTUS 123 59
MICROSOFT WORD 60

CHAPTER NINE ERROR HANDLING 61

INTRODUCTION 61
PROBLEMS TO ADDRESS 61
PROBLEM HANDLING 61

APPENDIX A DATA STRUCTURES 65

ALPHADDE PARAMETER LIMITS 65
DDE "SYSTEM" TOPIC ITEMS 65
ALPHABASIC 65
ASSEMBLER 66
ALPHADDE MESSAGE HEADER FORMAT 67

APPENDIX B ERROR CODES 69

APPENDIX C ALPHADDE MESSAGE TYPES 71

SOCKS LOGON/LOGOFF 71
TOPIC LOGON/LOGOFF 71
ITEM REQUEST 71
ITEM RESPONSE 72
MESSAGE FORMAT ERROR 73
DESTINATION UNKNOWN 73

APPENDIX D FUNCTIONAL DESCRIPTIONS 75

FUNCTIONAL DESCRIPTIONS OF ALPHADDE.EXE AND SOCKS.LIT 75

ALPHADDE.EXE 75
SOCKS.LIT 75

APPENDIX E ALPHADDE.INI 77

ENTRIES IN ALPHADDE.INI 77
EXAMPLE ALPHADDE.INI FILE: 78

APPENDIX F SETTING UP ALPHADDE 79

SETUP CONFIGURATIONS: 79
ALPHADDE INSTALLER INFORMATION: 79
ALPHADDE CHECK LIST 79
POINT TO POINT TEST 81

INDEX 82

Chapter One
Introduction

This manual describes AlphaDDE and how to use it to enhance your applications. AlphaDDE is a Client/Server interface between Microsoft Windows and AMOS. It has two main components: programs that link the two systems, and a developer's Toolkit for using that link. This manual contains descriptions of the library functions and messages involved in AlphaDDE, how to use them, and some information on setting up applications' DDE links. Components of the Toolkit include program libraries, sample code, and documents such as this.

AlphaDDE is a client/server communications link and developer's Toolkit. A client application (a program running in a window) on a PC sends a request to a server application (a program on the PC), and the server sends a response. To get this response, the server program sends a message to the AMOS host and receives a response from another server running there.

The client application can be almost any mainstream Windows application (such as Microsoft Excel or Word, Lotus 123, WordPerfect, etc.). The AMOS server is written by a developer (using this Guide) to return whatever information he or she wishes to provide.

AlphaDDE provides Microsoft Windows programs using standard DDE (Dynamic Data Exchange) messages with the ability to access resources on an AMOS system. Such resources are provided under the control of Topic Server programs, which respond to DDE requests for data. The primary purpose of this Guide is to provide you with the ability to write Topic Servers that can integrate PC functions into your applications.

Components of AlphaDDE

- ALPHADDE.EXE, a Windows program that acts as a DDE server on the PC, and communicates with AMOS systems. ALPHADDE.EXE runs minimized on the Windows desktop.
- SOCKS.LIT, an AMOS program that translates DDE messages received from AlphaTCP into ITC messages passed to the Topic Servers. SOCKS is PIC-encoded for a specific number of users. SOCKS runs continuously on the AMOS server.
- Library functions to make writing Topic Servers easier (and to make them more consistent.)
- Sample Topic Servers. These are provided with source code in AlphaBASIC and assembly. These are provided as templates for application developers.
- Application Usage. Some sample Visual Basic programs that illustrate how to write such applications will be provided. Also, information on how to use the DDE capabilities of Microsoft Word and Access, as well as other common Windows applications, will be provided. We will show how you can access AMOS data from these standard Windows applications.

Chapter Two
Installation and Configuration Notes

This chapter describes how to install and configure AlphaDDE, as well as setting up and testing the communications link between the two systems.

Requirements for AlphaDDE

- WINSOCK.DLL, a standard Windows TCP/IP protocol stack for the PC. Only the protocol stack is needed; TCP/IP applications such as Telnet and FTP are not, which can reduce the cost and complexity of the installation. WINSOCK.DLL may be obtained from any of a number of vendors. TCP/IP needs to be installed and running for AlphaDDE to function.
- AlphaTCP. TCP/IP for the Alpha Micro. Only a minimal configuration is necessary.
- AMOS 2.2C or later.
- AlphaNET. This is required for the Name Registry used by SOCKS and the Topic Servers (it is now included in AlphaTCP).
- Windows 3.1, Windows for Workgroups 3.11, or Windows 95. PCs that run AlphaDDE must run Windows in standard or enhanced (not real) mode.

Installation

The easiest installation uses the checklists in the appendix.

PC

1. A Winsock-compliant version of TCP/IP (implemented using WINSOCK.DLL) should be installed and configured on the PC.
2. Run the Setup program. This will create a directory called ALPHADDE on the PC and copy files to it. You will be asked to enter system network address information, and requested whether you want to install some optional modules. The Module Selections are:
   - AMOS Support Files—these are the files to be loaded on the AMOS system. You can load them here and copy them to the Alpha Micro (using AlphaLAN or other file transfer utility)
   - Documentation Files—are informational
   - Sample Code—are the Visual Basic and C sample programs described later in this document.
3. The Setup program adds AlphaDDE.exe to the Windows StartUp group. For the initial test, you can either restart Windows, or start AlphaDDE.exe directly from its StartUp icon.

AMOS

1. AMOS 2.2C or later, AlphaNET 2.3 or later, and AlphaTCP 1.1 or later should first be installed and running on the AMOS host.
2. Load the AlphaDDE files onto the AMOS system. The default PPN for the AlphaDDE Toolkit files is [7,55], ersatz DDE:.
3. Acquire a PIC for SOCKS.LIT and run SOCKS to enter it.
4. Add a job and pseudo TRMDEF for SOCKS to a test .INI file, to be started after TCPEMU. Reboot using this new .INI file, and verify that SOCKS is running (immediately after boot on slower systems, SOCKS may be in Ctrl/C for a short time while waiting for TCPEMU to initialize).
5. To test the installation, log to the AlphaDDE PPN and start SMPTOP.

Initial Test

Refer to the checklist in the Appendix. The test involves configuring and starting AlphaDDE.exe on the PC and SOCKS.LIT on the Alpha, and running the sample Topic Server SMPTOP.LIT on the Alpha as described above. Then, run the Installation Verification program from its icon in the AlphaDDE group in Windows.

Configuration Notes

Topic Servers run as stand-alone server jobs. To start them, you have several choices. These include:

- You can run the Topic Server in any normal job. This is what you normally do while writing, testing, or debugging a Topic Server. Once it is running, you probably want to use one of the next two methods.
- You can configure your Topic Servers in the system .INI file as SOCKS is done
- You can use the *start* command in AlphaTCP's CONFIG. file (reference the CONFIG. file in the AlphaTCP Administrator's Guide). Be aware that the *start* command uses a path that begins in TCP:.
- You can use the *startd* command (the command line version of the CONFIG. *start*). This is useful for testing, especially to see that you know how to set up the CONFIG. *start* command

The XCALLs that AlphaDDE uses to provide library services for AlphaBASIC are relatively large (over 6k). We recommend loading them into SYSTEM to conserve memory. This is particularly true at the time this document is being published, since there is a current problem with AlphaBASIC PLUS in handling large XCALL programs that are not preloaded.

Chapter Three
General Philosophy

DDE Concepts and Implementation

DDE (Dynamic Data Exchange) is a capability within Microsoft Windows for passing data between different programs. This data is passed in the form of messages according to a defined protocol. AlphaDDE uses a set of standard library functions called DDEML (DDE Management Library) to ensure that the DDE protocol is implemented consistently.

DDE is a standard Microsoft Windows message passing protocol. Most Windows applications support DDE. Many also support OLE (Object Linking and Embedding), which can communicate with DDE (in many ways OLE is a superset of DDE).

There are two programs involved in passing a DDE message. A Client program requests that the Server program process a given data item. Most applications involve passing data from the Server to the Client, but other actions are possible, as described below. Some applications refer to the Server as the *Source* and the Client as the *Destination*.

In DDE, data is addressed in three components. The Service or Application is usually the program name of the server application that responds to a given type of DDE message. The Topic specifies a particular set of information that the Service supports. The Item specifies a particular data unit. As an example, STQUOTE could be the service name of a stock quote program that uses the Topic to identify a stock exchange and the Item to specify a stock's ticker symbol. These parameters would be passed from the Client program to the Server, which would then return the current price of the specified stock.

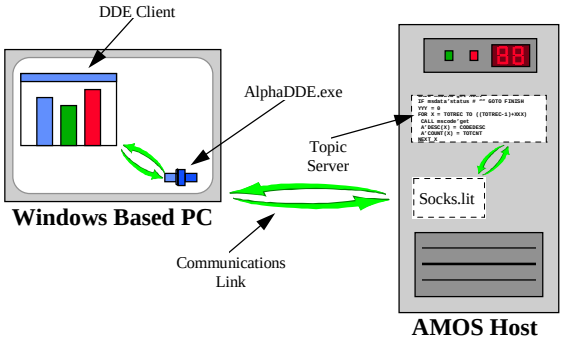
The types of actions that are available to DDE Client applications are:

- Requests for data from the Server. These fall into three categories:
  - an request for an Item to be sent immediately. This is called a Cold Link or item request.
  - a request to initiate a Hot Link. In this mode, the Server sends the Item to the Client whenever it changes in value. This is also called an Advise Link.
  - a request to initiate a Warm Link. In this mode, similar to a hot link but generally used for Items that are very large, the Server sends a notification that the Item has changed, after which the Client must request a copy of the data
- Sending a message to the Server. This is called a Poke.
- Sending a command to the Server. This is called an Execute message.

AlphaDDE Message Flow

AlphaDDE provides a data connection between an application on a PC running Microsoft Windows and a program called a Topic Server running on an AMOS system. The PC application uses a DDE (Dynamic Data Exchange) link to pass data to or from a DDE server program called. ALPHADDE.EXE passes this transaction request to the AMOS system, where a program called SOCKS.LIT passes it to a Topic Server program. SOCKS knows which Topic Server should get a given message because each Topic Server registers a given Topic with SOCKS. The Topic Server sends a response to SOCKS, which passes it back to the PC, which returns it to the original ("client") application.

This message flow is handled by AlphaDDE’s communications link. You as a developer need only be concerned with writing Topic Servers using the libraries provided, and possibly with DDE on the PC if writing new PC applications.



Developer Requirements

To use AlphaDDE, a developer writes a Topic Server. The Topic Server responds to item requests for data by doing some processing to obtain that data, and sending a reply. Sample Topic Servers are provided to demonstrate how they are written.

A developer may also wish to write applications (in a language such as C or Visual Basic) in Windows. However, AlphaDDE is designed such that standard Windows applications (such as Microsoft Word or Excel, or Quatro Pro) can be used to communicate with a Topic Server. This is one advantage of using DDE as the PC communications protocol.

A Note on Names

According to Visual Basic documentation, the *Service*, *Topic*, and *Item* are case-insensitive. However, this does not seem to be the case with all Windows applications. We recommend that you be consistent in your case usage to avoid problems.

Topic Servers

As mentioned before, the component programs of AlphaDDE are AlphaDDE.EXE on the PC and SOCKS.LIT and Topic Server programs located on an AMOS host running AlphaTCP. The primary function of the AlphaDDE Toolkit, and this Guide, is to provide the information necessary to be able to create your own Topic Servers. This section describes the functions contained in a Topic Server. Note that most of the standard functions described are implemented in library routines that are provided as part of the Toolkit.

Descriptions of ALPHADDE.EXE and SOCKS.LIT are provided in the appendix. This information is provided for reference only; it is not needed to be able to write Topic Servers.

Functional Description of Topic Servers

The Topic Server programs are ITC (AMOS Inter-Task Communications) applications that actually provide the DDE service functionality on the AMOS system. They could be jobs spawned by AlphaDDE from entries in **TCP:CONFIG.**, or could be continuously running jobs started by an .INI file.

The functions of Topic Servers are:

1. Initialize local AlphaDDE variables and connect to the AlphaDDE subsystem.

   Register their Topic(s) with the AMOS DDE interface (SOCKS). This involves registering their Topic(s) with the Name Registry using the name DDE*topicname*, finding SOCKS in the Name Registry, and sending a message for each Topic to SOCKS. The *Connect* function includes all these functions, as well as verifying parameters and doing other setup.

   The Topic message must be maintained so it can be re-sent on demand to SOCKS, which would be in the event SOCKS has been restarted. This is also the reason for registering as Topic Servers. Any received ITC message may be a SOCKS logon/logoff as well as Item Requests (see AlphaDDE Message Formats, later). The *Message Get* and *Message Process* functions will automatically recognize and respond properly to such messages.

2. Receive and process Item requests. One or more of the different types of DDE messages must be supported - Requests, Pokes, Advise Start, Advise Stop, or Execute. You may specify in the Commands Supported field of the *Connect* parameters which messages you'll accept, so that others will be rejected for you.

   The developer's work is to take a DDE message and respond to it — read a record, generate a report based on the request, or whatever else you wish.

3. Respond to the request using the *Send Response* function. **All Item requests require a response**. The response may be some requested data, a message that says "yes, I got that", or an error message.

   The received message (or at least its return address) must be saved for each Advise request. Note that both fields DDM.PC and DDM.WIN are required to fully identify a DDE Client.

4. For Advise (Hot/Warm) links, check for updated data. This type of function is used to automatically update data on a client window. When the target data changes, use the *Send Advise Data Ready* function to inform the client of this. If ready, said client will issue a item request for the updated data.

5. The *Save Advise Data* and *Advise Data Ready* functions are provided to do the housekeeping necessary to handle Advise (Hot/Warm) links.

6. When shut down, Topic Servers should sent a shutdown message to SOCKS. This is handled, as well as other housekeeping, by the *Disconnect* function.

Chapter Four
Functional Groups

The following describes the functional groups of commands that comprise AlphaDDE. Since AlphaDDE covers a variety of languages we will be discussing the logical operations only. These operations will have a functional equivalent for each of the directly supported languages. A more complete description of each routine and its usage is presented later in this manual.

Setup

Functions that deal with setting up and terminating the AlphaDDE environment.

DDECON Connects to AlphaDDE subsystem. The Connect routine configures the Topic Server for use with AlphaDDE, and sends messages to connect to the subsystem. DDECON also registers the name of the service you are providing so requesters can access your services. Includes the following:

dde'connect	(basic)
DDECON.SBR	(xcall)
DDECON	(assembler)

DDEDIS Disconnects from AlphaDDE subsystem. Disconnect cleans up and logs off. Includes the following:

dde'disconnect	(basic)
DDEDIS.SBR	(xcall)
DDEDIS	(assembler)

Simple Messaging

The routines in this category are designed to make relatively simple functions easy to accomplish. Here is an example of the sequence necessary to process an incoming item request for data (originated, of course, by a DDE client application on the PC).

1. Issue DDEGTR. This waits for a message, checks to see that it is an DDE Item Request (processing any other messages "under the hood"), and returns status indicating that an item request was received. The field DDM.ITEM contains the Item name requested.
2. Do whatever processing you need to do to set up the data response.
3. Issue DDESNR with the address of the data to be returned, and the size of the data field. The size field can be set to special values (-2, -3) to indicate if, for instance, you wish to reject the Item requested.
4. Repeat

The functions in this category are as follows:

DDEGTR Get Item Request. Wait indefinitely for an incoming message (^C will abort). Once a message is received from the client DDEGTR verifies that the server application can accept the message type. DDEGTR refuses those messages that are not supported and doesn't return to the calling program. Only after the message is accepted and verified does DDEGTR return to the calling application. Includes the following:

dde'wait'request	(basic)
DDEGTR.SBR	(xcall)
DDEGTR	(assembler)

DDESNR Send Item Response. After processing the item requested from the client program the server application will need to send some form of confirmation, rejection or data back to the client. DDESNR allows the application to application to do this. Includes the following:

dde’return’string	(basic)
dde’return’unf	(basic)
DDESNR.SBR	(xcall)
DDESNR	(assembler)

Messaging Functions

These functions deal with processing incoming messages with more control (and more responsibility) from the application program. Rather than unlimited time outs and complete message pre-filtering, these functions pass any message on to the application. Set time outs are defined during message wait. In this way the application can wait for a message, but get return back at pre-set intervals even if a message has not been received from a client process.

The following functions are actually combined into the DDEGTR (above) when time-outs or special message handling is not required. That ease of use sacrifices flexibility. The routines in this group split up the receive functions.

DDEGET Get ITC Message w/DDE status. DDEGET receives a message, whether DDE-format or not, to allow processing other ITC messages. It also returns more types of error information, and allows other processing if no data was received. If no message is received then a time-out condition can occur at defined intervals. Includes the following:

dde’get’request	(basic)
DDEGET.SBR	(xcall)
DDEGET	(assembler)

DDEPRC Process and Verify DDE Request is Valid. DDEPRC does the data verification functions of DDEGTR. Includes the following:

dde’process’request	(basic)
DDEPRC.SBR	(xcall)
DDEPRC	(assembler)

Hot (Advise) Link Support

These routines ease the handling of Hot links. DDESVA saves the information necessary to identify a particular DDE client request (PC, client window, Item, data format, etc.). DDESNA sends an "Advise" message to the client to indicate that updated data is available. The client will send an item request for the updated data if it so desires. Note that within AlphaDDE, all hot links are treated as warm links (the actual difference is in how messages are handled in the PC).

DDESVA Save Advise Information (for Hot links). This is called when processing an Advise Start request passed by DDEPRC. Includes the following:

dde’add’hotlnk	(basic)
dde’delete’hotlnk	(basic)
DDESVA.SBR	(xcall)
DDESVA	(assembler)

DDESNA Send Advise Data Ready (Hot link data changed). This is sent by a Topic Server to the client window to tell it that the data has changed, so the client should make an item request for the new data. Includes the following:

dde’advise’all	(basic)
DDESNA.SBR	(xcall)

DDESNA (assembler)

Chapter Five
AlphaDDE Function Reference Sheets

The following pages describe in detail the command set that makes up AlphaDDE. The various types of commands are included with their specific style listed to the right of the function name. This designation is defined as:

(amos)	This is an AMOS level command.
(assembler)	This is an Assembly Language routine. Libraries included with AlphaDDE will be linked to provide access to these routines
(basic)	This is an AlphaBASIC subroutine and are used as the high level access to AlphaDDE. These functions are defined in INCLUDE files that are part of the AlphaDDE tool kit.
(xcall)	These are AlphaBASIC and AlphaBASIC PLUS xcalls that are required to support the AlphaBASIC AlphaDDE tool kit. Their functional descriptions are provided for your information. These routines should be used for low level support or when the high level (basic) routines are inadequate. This chapter lists AlphaBASIC .SBR's; there is a full set of equivalent .XBRs for AlphaBASIC PLUS users.

dde’add’hotlnk (basic)

This routine adds a new entry to the “Hot Link” table. When a client job requests “Advise Start” (*dhd’flg = dde’fswrmI*) the application adds the client to the advise list for future updates. When the data being requested changes in the server application, it will advise all the clients in the list of this situation (via **dde’advise’all**.) All the information necessary to track the client is stored in the table.

Libraries

++INCLUDE DDE.MAP

++INCLUDE DDE.BSI

Prerequisite

Connection to AlphaDDE

Message from client via **dde’process’request** that indicates a request for “Advise Start”

Syntax

```
CALL  dde’add’hotlnk
```

Input

None

Output

dde’hotcnt Total clients in the list

Comments

The default maximum number of entries in the client list is 50. After this number of simultaneous clients is reached, future clients will receive an Advise Reject to indicate a full condition.

Use **dde’delete’hotlnk** to remove clients as they drop off (*dhd’flg = dde’fendl*.)

Example

```
++INCLUDE DDE.MAP

LOOP: CALL  CHECK'DATA              ! Check for changed data

LOOP1:PRINT
      PRINT "<get>";
      CALL  dde'get'request               ! Wait for message
      IF    dde'status # 0                      &
            PRINT "<timeout>";                  : &
            GOTO  LOOP

      PRINT "<process>";
      CALL  dde'process'request           ! Process this message
      IF    dde'status # 0                      &
            GOTO  LOOP1

      ! Is it a request to be added to our advise table?
      IF    dhd'flg = dde'fswrm                 &
            PRINT "<add>";                      : &
            CALL  dde'add'hotlnk

      GOTO  LOOP1

++INCLUDE DDE.BSI
```

See Also

dde’delete’hotlnk, dde’advise’all

Synonym

DDESVA(xcall), DDESVA(assembler)

dde’advise’all (basic)

This routine advises everyone in the advise client list that they must request data. All the clients that have been added (via **dde’add’hotlnk**) will be notified. This will initiate an item request for data from the client. *All* of the members of the list will be notified.

Libraries

++INCLUDE DDE.MAP

++INCLUDE DDE.BSI

Prerequisite

Connection to AlphaDDE

Syntax

```
CALL  dde’advise’all
```

Input

None

Output

None

Comments

All the clients, regardless of item type, will be informed of the change. To select only those members of a certain item type the application must search the list (see DDE.MAP for details of the list structure) for the appropriate clients requesting a particular item.

Example

```
++INCLUDE DDE.MAP

! Check our database for any changes, if so advise clients.
!
      IF    dde'hotcnt = 0                      &
            RETURN

      PRINT "<advise>";
      CALL  dde'advise'all          ! For the sample we always advise
      RETURN

++INCLUDE DDE.BSI
```

See Also

dde’add’hotlnk, dde’delete’hotlnk

Synonym

DDESNA(xcall), DDESNA(assembler)

dde’connect (basic)

This routine is designed to establish communications with AlphaDDE. In doing so the services provided by this program are registered to the network. Various parameters are also configured to indicate the type of messages, time-outs, attributes, etc. that this server program can handle or requires.

Libraries

++INCLUDE DDE.MAP

++INCLUDE DDE.BSI

Prerequisite

None

Syntax

```
CALL  dde’connect
```

Input

dde’servn The Service name to be used , or “” to use the default of “AlphaDDE”. *ONLY “AlphaDDE” IS SUPPORTED AT THIS TIME*; use “” for this parameter.

dde’topicn String of the Topic name to be used.

dde’formats(x) An list of up to ten (10) data formats that this Topic Server supports (see DDE.FMT in DDEDEF.M68). A value of 0 will support only the default format of CF_TEXT (ASCIZ). ONLY CF_TEXT (ASCIZ) IS SUPPORTED AT THIS TIME; use 0 for this parameter.

dde’cmdsup Is a two-byte bit mask of commands that this Topic Server can process (others will be rejected). 0 IS INVALID (implies no commands are supported). Add together any of the following:

- dde'crqst Can support item request for data.
- dde'cpoke Can support poke data.
- dde'cadv Can support advise requests (hot link).
- dde'cexec Can support execute

dde’msgwait Is the amount of time (in 1/100’s of a second increments) that DDEGET (see below) will wait for a message. 0 = wait forever for message.

dde’dspflg Are flags that can be set to output different types of informative messages on the terminal. These are:

- dds’verbos Displays an ASCII message for any errors, and outputs messages indicating actions that occur within the Topic Server
- dds’info Decodes AlphaDDE and ITC messages

The **bolded** variables above signify required arguments. All others are optional.

Output

dde’status Status of the call. 0 is no error. See appendix for additional error codes.

dde’servn Service name, (“AlphaDDE” ONLY FOR NOW)

dde’formats(x) List of up to ten (10) words of data formats supported (ONLY CF_TEXT = ASCIZ SUPPORTED NOW).

dde’myaddr Current system's CPU ID

dde’mysock current job's socket number

dde’socksa CPU ID of SOCKS (if found via Name Registry)

dde’sockss Job socket number of SOCKS (if found in)

dde’topbuf Topic Logon message

Comments

This call *must* be made upon initialization of the program and prior to any other AlphaDDE calls. Call this routine only once within the bounds of a single program. Once communications are established you can perform any of the AlphaDDE commands. Upon termination from the program you should call DDEDIS.SBR to inform AlphaDDE that this service is no longer available.

Example

```
++INCLUDE DDE.MAP
      dde'topicn  = "SampleTopic"
      dde'cmdsup  = dde'crqst
      CALL        dde'connect
      IF    dde'status # 0    GOTO quit
            \    /
            /    \

++INCLUDE DDE.BSI
```

See Also

dde'disconnect

Synonym

DDECON(xcall), DDECON (assembler)

dde’delete’hotlnk (basic)

This routine deletes an entry from the “Hot Link” table. When a client job requests “Advise Stop” (*dhd’flg = dde’fendI*) the application should remove the client from the advise list using this command.

Libraries

++INCLUDE DDE.MAP

++INCLUDE DDE.BSI

Prerequisite

Connection to AlphaDDE

Message from client via **dde’process’request** that indicates a request for “Advise Stop”

Syntax

```
CALL  dde’delete’hotlnk
```

Input

None

Output

dde’hotcnt Total clients in the list

Comments

The advise list is checked for this clients entry, once found it is removed for any future advises.

Example

```
++INCLUDE DDE.MAP

LOOP: CALL  CHECK'DATA              ! Check for changed data

LOOP1:
      PRINT
      PRINT "<get>";
      CALL  dde'get'request               ! Wait for message
      IF    dde'status # 0                      &
            PRINT "<timeout>";                  : &
            GOTO  LOOP

      PRINT "<process>";
      CALL  dde'process'request           ! Process this message
      IF    dde'status # 0                      &
            GOTO  LOOP1

      ! Is it a request to be removed from our advise table?
      IF    dhd'flg = dde'fendl                 &
            PRINT "<delete>";                   : &
            CALL  dde'delete'hotlnk

      GOTO        LOOP1             ! Wait for the next message

++INCLUDE DDE.BSI
```

See Also

dde’add’hotlnk, dde’advise’all

dde'disconnect (basic)

Disconnect from AlphaDDE subsystem. This routine is usually called after a CTRL/C or other termination. It sends a Topic Logoff message to SOCKS, removes itself from the, and does other cleanup as necessary.

Libraries

++INCLUDE DDE.MAP

++INCLUDE DDE.BSI

Prerequisite

Connection to AlphaDDE (dde'connect)

Syntax

```
CALL  dde'disconnect
```

Input

None

Output

None

Comments

This call *must* be made upon termination of the program. Call this routine only once within the bounds of a single program. This call informs AlphaDDE that this service is no longer available.

Example

```
++INCLUDE DDE.MAP
      dde'topicn  = "SampleTopic"
      dde'cmdsup  = dde'crqst
      CALL        dde'connect
      IF    dde'status # 0    GOTO quit
            \    /
            /    \
      CALL        dde'disconnect
      END

++INCLUDE DDE.BSI
```

See Also

dde'disconnect

Synonym

DDECON(xcall), DDECON (assembler)

dde'get'request (basic)

This routine checks for a pending ITC message, and waits for up to MSGWAIT for one. If a message is found, it is checked to see if it is an AlphaDDE message.

Libraries

++INCLUDE DDE.MAP

++INCLUDE DDE.BSI

Prerequisite

Connection to AlphaDDE

Syntax

```
CALL  dde'get'request
```

Input

None

Output

dde'status	Status of the call. 0 is no error. See appendix for additional error codes.
dde'servn	Service name, ("AlphaDDE" ONLY FOR NOW)
dde'formats(x)	List of up to ten (10) words of data formats supported (ONLY CF_TEXT = ASCIZ SUPPORTED NOW).
dde'myaddr	Current system's CPU ID
dde'mysock	current job's socket number
dde'socksa	CPU ID of SOCKS (if found via Name Registry)
dde'sockss	Job socket number of SOCKS (if found in Name Registry)
dde'topbuf	Topic Logon message
dde'flg	Transaction request. Which is one of the following:
dde'fsend (0)	Request for server to send data ("Cold Request")
dde'fpoke (1)	Request for send data, with "Poke Data"
dde'fshot (2)	Request for server to start "Hot Link" with this client.
dde'fswrm (3)	Request for server to start "Warm Link" with this client.
dde'endl (4)	Terminate Warm or Hot Link with this client.
dde'fexec (5)	Execute
dde'fmt	Data format requested
dhd'bsd	Identifier of PC for the Response message
dhd'soc	Identifier of window for Response message
dde'service	Service name of the message received
dde'topic	Topic name of the message received
dde'item	Item name of the message received

Comments

This command will wait up to a maximum of the time out period defined in *dde'msgwait*. Control will be returned to the calling program immediately following an incoming message or the time our period.

This function is most useful in Hot Link servers where the application program must wake up occasionally to check for a change in the data being requested.

To filter unwanted messages call **DDEPRC** after determining that a something other than a time out occurred.

DDEGET and **DDEPRC** are combined in the higher level **DDEGTR** function. Although **DDEGTR** does not offer the flexibility of the discreet functions, implementation is much simpler.

Example

```
++INCLUDE DDE.MAP

LOOP: CALL  CHECK'DATA              ! Check for changed data

LOOP1:
      PRINT
      PRINT "<get>";
      CALL  dde'get'request               ! Wait for message
      IF    dde'status # 0                      &
            PRINT "<timeout>";                  : &
            GOTO  LOOP

      PRINT "<process>";
      CALL  dde'process'request           ! Process this message
      IF    dde'status # 0                      &
            GOTO  LOOP1

      ! Is it a request for data?
      IF    dhd'flg = dde'fsend                 &
            CALL  SEND'DATA

++INCLUDE DDE.BSI
```

See Also

dde’wait’request

Synonym

DDEGET(xcall), DDEGET(assembler)

dde'process'request (basic)

This routine checks to see if the requested command ("Cold Request", Start Hot Link, Poke, etc.) is supported by this Topic Server, based on the CMDSUP parameter to DDECON. It also verifies that the message is in the proper format and that the Service (MUST BE AlphaDDE AT PRESENT) and Topic are correct. It also checks that the data format requested is supported by this Topic Server. If the message passes all such checks, the Item Request command and data format are returned. If a message fails any check, an Item Response message is sent with the appropriate error code.

Libraries

++INCLUDE DDE.MAP

++INCLUDE DDE.BSI

Prerequisite

Connection to AlphaDDE

Message from client via **dde'get'request**

Syntax

```
CALL  dde'process'request
```

Input

None

Output

dde'status	Status of the call. 0 is no error. See appendix for additional error codes.
dde'servn	Service name, ("AlphaDDE" ONLY FOR NOW)
dde'formats(x)	List of up to ten (10) words of data formats supported (ONLY CF_TEXT = ASCIZ SUPPORTED NOW).
dde'myaddr	Current system's CPU ID
dde'mysock	current job's socket number
dde'socksa	CPU ID of SOCKS (if found via Name Registry)
dde'sockss	Job socket number of SOCKS (if found in Name Registry)
dde'topbuf	Topic Logon message
dde'flg	Transaction request. Which is one of the following:
dde'fsend (0)	Request for server to send data ("Cold Request")
dde'fpoke (1)	Request for send data, with "Poke Data"
dde'fshot (2)	Request for server to start "Hot Link" with this client.
dde'fswrm (3)	Request for server to start "Warm Link" with this client.
dde'endl (4)	Terminate Warm or Hot Link with this client.
dde'fexec (5)	Execute
dde'fmt	Data format requested
dhd'bsd	Identifier of PC for the Response message
dhd'soc	Identifier of window for Response message
dde'service	Service name of the message received
dde'topic	Topic name of the message received
dde'item	Item name of the message received

Comments

dde'get'request and **dde'process'request** are combined in the higher level **dde'wait'request** function. Although **dde'wait'request** does not offer the flexibility of the discreet functions, implementation is much simpler.

Example

```
++INCLUDE DDE.MAP

LOOP: CALL  CHECK'DATA              ! Check for changed data

LOOP1:
      PRINT
      PRINT "<get>";
      CALL  dde'get'request               ! Wait for message
      IF    dde'status # 0                      &
            PRINT "<timeout>";                  : &
            GOTO  LOOP

      PRINT "<process>";
      CALL  dde'process'request           ! Process this message
      IF    dde'status # 0                      &
            GOTO  LOOP1

      ! Is it a request for data?
      IF    dhd'flg = dde'fsend                 &
            CALL  SEND'DATA

++INCLUDE DDE.BSI
```

See Also

dde'get'request

Synonym

DDEPRC(xcall), DDEPRC(assembler)

dde’return’string & dde’return’unf (basic)

This routine sends a DDE Item Response message to the client based on the size/flags field. If the size field is greater than or equal to 0, the data buffer parameter points to data to be returned to a PC window. If the size field is less than 0, it is a flag value indicating what non-data response is to be sent.

Libraries

++INCLUDE DDE.MAP

++INCLUDE DDE.BSI

Prerequisite

Connection to AlphaDDE

Item request for response from client

Syntax

```
CALL  dde’return’string              ! For simple text data
CALL  dde’return’unf                 ! For unformatted data
```

Input

dde’rtn’string For text data use this variable for return data

or

dde’rtn’unf For unformatted data use this variable for return data

dde’size The size of the data to return to the client or one of the following:

dde’scalc (0) Calculate the size of the text string (or 1024 bytes if unformatted)

dde’sack (-1) if a Poke is to be acknowledged

dde’sbad (-2) if an Item not found message is to be sent

dde’sstat (-3) if *dhd’stat* and *dhd’flg* contain status information to return

dde’sadack (-4) if an Advise Request (Start or Stop) is to be acknowledged

dde’sadrej (-5) if an Advise Request is to be rejected

Output

dde’status Status of the call. 0 is no error. See appendix for additional error codes.

Comments

This command allows the application program to return data and information back to the client. This command should only be used after a request (DDEGTR or DDEGET) has been received from the client. At that point the client has requested data and is ready for a response.

In the case of a hot link, when the application determines that the data has been updated and needs to be sent to the client use DDESNA. This command will advise the client that data is ready. The client will then request the data. This request will be accepted through (DDEGTR or DDEGET.)

Example

```
++INCLUDE DDE.MAP
      IF    dhd'flg = dde'fsend                                  &
            PRINT "<send, item: ";CHR$(34);dde'item;CHR$(34);">"; : &
            X           = X + 1
            dde'rtn'string    = "This is message "+STR(X)
            CALL        dde'return'string
++INCLUDE DDE.BSI
```

See Also

dde’get’request, dde’wait’request

Synonym

DDEGTR(xcall), DDEGTR(assembler)

dde'wait'request (basic)

This routine checks for a pending ITC message, and waits up to a Ctrl-C. If a message is found, it is checked to see if it is a properly formatted AlphaDDE message. If the message passes all checks, the Item Request command and data format are returned (in *dhd'flg* and *dhd'fmt* in DDE.MAP), and the Item itself is returned in dde'item.

Libraries

++INCLUDE DDE.MAP

++INCLUDE DDE.BSI

Prerequisite

Connection to AlphaDDE

Syntax

```
CALL  dde'wait'request
```

Input

None

Output

dde'status	Status of the call. 0 is no error. See appendix for additional error codes.
dde'servn	Service name, ("AlphaDDE" ONLY FOR NOW)
dde'formats(x)	List of up to ten (10) words of data formats supported (ONLY CF_TEXT = ASCIZ SUPPORTED NOW).
dde'myaddr	Current system's CPU ID
dde'mysock	current job's socket number
dde'socksa	CPU ID of SOCKS (if found via Name Registry)
dde'sockss	Job socket number of SOCKS (if found in Name Registry)
dde'topbuf	Topic Logon message
dde'flg	Transaction request. Which is one of the following:
dde'fsend (0)	Request for server to send data ("Cold Request")
dde'fpoke (1)	Request for send data, with "Poke Data"
dde'fshot (2)	Request for server to start "Hot Link" with this client.
dde'fswrm (3)	Request for server to start "Warm Link" with this client.
dde'endl (4)	Terminate Warm or Hot Link with this client.
dde'fexec (5)	Execute
dde'fmt	Data format requested
dhd'bsd	Identifier of PC for the Response message
dhd'soc	Identifier of window for Response message
dde'service	Service name of the message received
dde'topic	Topic name of the message received
dde'item	Item name of the message received
dde'rtn'string	If this is a poke message then this variable contains the clients poke data.

Comments

AlphaDDE "control" messages such as SOCKS Logon or Logoff are processed and returned. Otherwise, if the command ("Cold" Request, Start Hot Link, Poke, etc.) is not supported by this Topic Server, based on the CMDSUP parameter to DDECON, an error message is returned. The message is verified to be properly formatted and that the Service (MUST BE AlphaDDE AT PRESENT) and Topic are correct. It also checks that the data format requested is supported by this Topic Server. If an AlphaDDE message fails

any check, an Item Response message is sent with the appropriate error code. If a message is received that is not in AlphaDDE format, it is ignored. If you need more control than this, see DDEGET.

Example

```
++INCLUDE DDE.MAP

      CALL        dde'wait'request
      IF    dde'status # 0    GOTO quit
            \    /
            /    \

++INCLUDE DDE.BSI
```

See Also

dde'get'request, dde'process'request

Synonym

DDEGTR(xcall), DDEGTR(assembler)

DDECON.SBR (xcall)

This routine is designed to establish communications with AlphaDDE. In doing so the services provided by this program are registered to the network. Various parameters are also configured to indicate the type of messages, time-outs, attributes, etc. that this server program can handle or requires.

Libraries

++INCLUDE DDE.MAP *(optional)*

Prerequisite

None

Syntax

XCALL DDECON, dde, dde'status

dde is an AlphaBASIC structure defined in the appendix (this structure is defined in DDE.MAP).

dde'status is the optional second argument and can be of any type. It is the return status of the call. 0 is no error. See appendix for additional error codes.

Comments

This call *must* be made upon initialization of the program and prior to any other AlphaDDE calls. Call this routine only once within the bounds of a single program. Once communications are established you can perform any of the AlphaDDE commands. Upon termination from the program you should call DDEDIS.SBR to inform AlphaDDE that this service is no longer available.

This function works on all versions of AlphaBASIC.

DDECON.SBR is reentrant and reusable and may be loaded into either system or user memory.

Example

```
++INCLUDE DDE.MAP
      XCALL       SETB4, 0, dde'display
      dde'topicn  = "ABTEST"
      dde'cmdsup  = dde'crqst + dde'cpoke
      XCALL       DDECON, dde, dde'status
```

See Also

DDEDIS.SBR, SETB4.SBR

Synonym

dde'connect(basic), DDECON (assembler)

DDEDIS.SBR (xcall)

Disconnect from AlphaDDE subsystem. This routine is usually called after a CTRL/C or other termination. It sends a Topic Logoff message to SOCKS, removes itself from the Name Registry, and does other cleanup as necessary.

Libraries

++INCLUDE DDE.MAP *(optional)*

Prerequisite

None

Syntax

XCALL DDEDIS, dde

dde is an AlphaBASIC structure defined in the appendix (this structure is defined in DDE.MAP).

Comments

This call *must* be made upon termination of the program. Call this routine only once within the bounds of a single program. This call informs AlphaDDE that this service is no longer available.

This function works on all versions of AlphaBASIC.

DDEDIS.SBR is reentrant and reusable and may be loaded into either system or user memory.

Example

```
++INCLUDE DDE.MAP
      XCALL       SETB4, 0, dde'display
      dde'topicn  = "ABTEST"
      dde'cmdsup  = dde'crqst + dde'cpoke
      XCALL       DDECON, dde, dde'status
            \    /
            /    \
      XCALL       DDEDIS, dde
      END
```

See Also

DDECON.SBR

Synonym

dde'disconnect(basic), DDEDIS (assembler)

DDEGET.SBR (xcall)

This routine checks for a pending ITC message, and waits for up to MSGWAIT for one. If a message is found, it is checked to see if it is an AlphaDDE message.

Libraries

`++INCLUDE DDE.MAP` *(optional)*

Prerequisite

Connection to AlphaDDE

Syntax

```
XCALL DDEGET dde{, status}
```

`dde`	Is an AlphaBASIC structure defined in the appendix (this structure is defined in DDE.MAP).
`status`	Return status of the call.

Input

None

Output

`status`	0 if an AlphaDDE message was received. -1 if Ctrl/C occurred. See appendix for additional codes.
dde'servn	Service name, ("AlphaDDE" ONLY FOR NOW)
dde'formats(x)	List of up to ten (10) words of data formats supported (ONLY CF_TEXT = ASCIZ SUPPORTED NOW).
dde'myaddr	Current system's CPU ID
dde'mysock	current job's socket number
dde'socksa	CPU ID of SOCKS (if found via Name Registry)
dde'sockss	Job socket number of SOCKS (if found in Name Registry)
dde'topbuf	Topic Logon message
dde'flg	Transaction request. Which is one of the following:
dde'fsend (0)	Request for server to send data ("Cold Request")
dde'fpoke (1)	Request for send data, with "Poke Data"
dde'fshot (2)	Request for server to start "Hot Link" with this client.
dde'fswrm (3)	Request for server to start "Warm Link" with this client.
dde'endl (4)	Terminate Warm or Hot Link with this client.
dde'fexec (5)	Execute
dde'fmt	Data format requested
dhd'bsd	Identifier of PC for the Response message
dhd'soc	Identifier of window for Response message
dde'service	Service name of the message received
dde'topic	Topic name of the message received
dde'item	Item name of the message received

Comments

This command will wait up to a maximum of the time out period defined in *dde'msgwait*. Control will be returned to the calling program immediately following an incoming message or the time our period.

This function is most useful in Hot Link servers where the application program must wake up occasionally to check for a change in the data being requested.

To filter unwanted messages call **DDEPRC** after determining that a something other than a time out occurred.

DDEGET and **DDEPRC** are combined in the higher level **DDEGTR** function. Although **DDEGTR** does not offer the flexibility of the discreet functions, implementation is much simpler.

Example

```
++INCLUDE DDE.MAP

      XCALL       setb4,0,dde'display
      dde'topicn  = "ABTEST"
      dde'cmdsup  = dde'crqst+dde'cpoke
      XCALL       DDECON, dde, dde’status

LOOP: XCALL       DDEGET, dde, dde’status
      IF    dde’status # 0                      &
            GOTO  LOOP
      XCALL       DDEPRC, dde, dde’status, dde’rtn’string
      PRINT       “Item string = <“;dde'item;”>“
      IF    dhd’flg = dde’fpoke                 &
            PRINT “Poke string = <“;dde’rtn’string;”>“
```

See Also

DDEGTR, DDEPRC

Synonym

dde’get’request(basic), DDEGET(assembler)

DDEGTR.SBR (xcall)

This routine checks for a pending ITC message, and waits up to a Ctrl-C. If a message is found, it is checked to see if it is a properly formatted AlphaDDE message. If the message passes all checks, the Item Request command and data format are returned (in *dhd'flg* and *dhd'fmt* in DDE.MAP), and the Item itself is returned in dde'item.

Libraries

`++INCLUDE DDE.MAP` *(optional)*

Prerequisite

Connection to AlphaDDE

Syntax

```
XCALL DDEGTR dde{, status{, datast}}
```

`dde`	Is an AlphaBASIC structure defined in the appendix (this structure is defined in DDE.MAP).
`status`	Return status of the call.
`datast`	String containing return data

Input

None

Output

`status`	0 if an AlphaDDE message was received. -1 if Ctrl/C occurred. See appendix for additional codes.
`datast`	If the client message is a Poke Data then this string variable will contain the data sent from the client.
dde'servn	Service name, ("AlphaDDE" ONLY FOR NOW)
dde'formats(x)	List of up to ten (10) words of data formats supported (ONLY CF_TEXT = ASCIZ SUPPORTED NOW).
dde'myaddr	Current system's CPU ID
dde'mysock	current job's socket number
dde'socksa	CPU ID of SOCKS (if found via Name Registry)
dde'sockss	Job socket number of SOCKS (if found in Name Registry)
dde'topbuf	Topic Logon message
dde'flg	Transaction request. Which is one of the following:
dde'fsend (0)	Request for server to send data ("Cold Request")
dde'fpoke (1)	Request for send data, with "Poke Data"
dde'fshot (2)	Request for server to start "Hot Link" with this client.
dde'fswrm (3)	Request for server to start "Warm Link" with this client.
dde'endl (4)	Terminate Warm or Hot Link with this client.
dde'fexec (5)	Execute
dde'fmt	Data format requested
dhd'bsd	Identifier of PC for the Response message
dhd'soc	Identifier of window for Response message
dde'service	Service name of the message received
dde'topic	Topic name of the message received
dde'item	Item name of the message received

Comments

AlphaDDE "control" messages such as SOCKS Logon or Logoff are processed and returned. Otherwise, if the command ("Cold" Request, Start Hot Link, Poke, etc.) is not supported by this Topic Server, based on the CMDSUP parameter to DDECON, an error message is returned. The message is verified to be properly formatted and that the Service (MUST BE AlphaDDE AT PRESENT) and Topic are correct. It also checks that the data format requested is supported by this Topic Server. If an AlphaDDE message fails any check, an Item Response message is sent with the appropriate error code. If a message is received that is not in AlphaDDE format, it is ignored. If you need more control than this, see DDEGET.

Example

```
++INCLUDE DDE.MAP

      XCALL       setb4,0,dde'display
      dde'topicn  = "ABTEST"
      dde'cmdsup  = dde'crqst+dde'cpoke
      XCALL       DDECON, dde, dde’status

LOOP: XCALL       DDEGTR, dde, dde’status, dde’rtn’string
      PRINT       “Item string = <“;dde'item;”>“
      IF    dhd’flg = dde’fpoke                &
            PRINT “Poke string = <“;dde’rtn’string;”>“
```

See Also

DDESNR, DDEGET, DDEPRC

Synonym

dde’wait’request(basic), DDEGTR(assembler)

DDEPRC.SBR (xcall)

This routine checks to see if the requested command ("Cold Request", Start Hot Link, Poke, etc.) is supported by this Topic Server, based on the CMDSUP parameter to DDECON. It also verifies that the message is in the proper format and that the Service (MUST BE AlphaDDE AT PRESENT) and Topic are correct. It also checks that the data format requested is supported by this Topic Server. If the message passes all such checks, the Item Request command and data format are returned. If a message fails any check, an Item Response message is sent with the appropriate error code.

Libraries

`++INCLUDE DDE.MAP` *(optional)*

Prerequisite

Connection to AlphaDDE

Incoming message from DDEGET

Syntax

```
XCALL DDEPRC dde{, status{, datast}}
```

`dde`	Is an AlphaBASIC structure defined in the appendix (this structure is defined in DDE.MAP).
`status`	Return status of the call.
`datast`	String containing return data

Input

None

Output

`status`	0 if an AlphaDDE message was received. -1 if Ctrl/C occurred. See appendix for additional codes.
`datast`	If the client message is a Poke Data then this string variable will contain the data sent from the client.
dde'servn	Service name, ("AlphaDDE" ONLY FOR NOW)
dde'formats(x)	List of up to ten (10) words of data formats supported (ONLY CF_TEXT = ASCIZ SUPPORTED NOW).
dde'myaddr	Current system's CPU ID
dde'mysock	current job's socket number
dde'socksa	CPU ID of SOCKS (if found via Name Registry)
dde'sockss	Job socket number of SOCKS (if found in Name Registry)
dde'topbuf	Topic Logon message
dde'flg	Transaction request. Which is one of the following:
dde'fsend (0)	Request for server to send data ("Cold Request")
dde'fpoke (1)	Request for send data, with "Poke Data"
dde'fshot (2)	Request for server to start "Hot Link" with this client.
dde'fswrm (3)	Request for server to start "Warm Link" with this client.
dde'endl (4)	Terminate Warm or Hot Link with this client.
dde'fexec (5)	Execute
dde'fmt	Data format requested
dhd'bsd	Identifier of PC for the Response message
dhd'soc	Identifier of window for Response message

dde'service	Service name of the message received
dde'topic	Topic name of the message received
dde'item	Item name of the message received

Comments

DDEGET and **DDEPRC** are combined in the higher level **DDEGTR** function. Although **DDEGTR** does not offer the flexibility of the discreet functions, implementation is much simpler.

Example

```
++INCLUDE DDE.MAP

      XCALL       setb4,0,dde'display
      dde'topicn  = "ABTEST"
      dde'cmdsup  = dde'crqst+dde'cpoke
      XCALL       DDECON, dde, dde'status

LOOP: XCALL       DDEGET, dde, dde'status
      IF    dde'status # 0                      &
            GOTO  LOOP
      XCALL       DDEPRC, dde, dde'status, dde'rtn'string
      PRINT       "Item string = <";dde'item;">"
      IF    dhd'flg = dde'fpoke                 &
            PRINT "Poke string = <";dde'rtn'string;">"
```

See Also

DDEGTR, DDEGET

Synonym

dde'process'request(basic), DDEPRC(assembler)

DDESNA.SBR (xcall)

Send Advise Data Ready Message. After a hot link (Advise loop) has been initiated, you call this routine to indicate that there is new data. The client will issue a request for that data when he is ready.

Libraries

`++INCLUDE DDE.MAP` *(optional)*

Prerequisite

Connection to AlphaDDE

Syntax

```
XCALL DDESNA dde, dhl'link{, status}
```

`dde`	Is an AlphaBASIC structure defined in the appendix (this structure is defined in DDE.MAP).
`dhl'link`	Advise Link structure as defined in the appendix (this structure is defined in DDE.MAP as an element of an array of Advise Links.)
`status`	Return status of the call.

Input

`dhl'link`	Advise Link structure filled previously by **DDESVA.**

Output

`status`	0 if an AlphaDDE message was received. -1 if Ctrl/C occurred. See appendix for additional codes.

Comments

When creating a hot topic server any number of clients can request information simultaneously. For each request you must save the data block returned by **DDESVA** in some form of array. When the data updates the application will go through this array and inform the client (via **DDESNA**) that the data has changed.

Example

```
++INCLUDE DDE.MAP

dde'advise'all:
!
! Advise everyone in our table that the data has been updated
!
      IF    dde'hotcnt = 0                                  &
            RETURN

      FOR   dde'xxx = 1 TO dde'maxhot
       IF   dhl'inuse(dde'xxx)                              &
            XCALL DDESNA, dde, dhl'link(dde'xxx), dde'status

      NEXT  dde'xxx
      RETURN
```

See Also

DDESVA

Synonym

dde'advise'all(basic), DDESNA(assembler)

DDESNR.SBR (xcall)

This routine sends a DDE Item Response message to the client based on the size/flags field. If the size field is greater than or equal to 0, the data buffer parameter points to data to be returned to a PC window. If the size field is less than 0, it is a flag value indicating what non-data response is to be sent.

Libraries

`++INCLUDE DDE.MAP` *(optional)*

Prerequisite

Connection to AlphaDDE

Item request for data from client

Syntax

```
XCALL DDESNR dde, datbuf, size{, status}
```

`dde`	Is an AlphaBASIC structure defined in the appendix (this structure is defined in DDE.MAP).
`datbuf`	Send data string or unformatted.
`size`	Size of send data buffer.
`status`	Return status of the call.

Input

`datbuf`	Is the string or unformatted data to be sent back to the client program.
`size`	The size parameter defines the size of the data buffer or the response type. *size* is either the absolute data buffer size (for binary or text data) or one of the following:
dde'scalc (0)	if the data is ASCIZ (library to calculate size)
dde'sack (-1)	if a Poke is to be acknowledged
dde'sbad (-2)	if an Item not found message is to be sent
dde'sstat (-3)	if *dhd'stat* and *dhd'flg* contain status information to return
dde'sadack (-4)	if an Advise Request (Start or Stop) is to be acknowledged
dde'sadrej (-5)	if an Advise Request is to be rejected
dde'service	Service name for the Response message
dde'topic	Topic name for the Response message
dde'item	Item name for the Response message
dhd'bsd	Identifier of PC for the Response message
dhd'soc	Identifier of window for Response message

Output

`status`	0 if an AlphaDDE message was received. -1 if Ctrl/C occurred. See appendix for additional codes.

Comments

This command allows the application program to return data and information back to the client. This command should only be used after an item request (DDEGTR or DDEGET) has been received from the client. At that point the client has requested data and is ready for a response.

In the case of a hot link, when the application determines that the data has been updated and needs to be sent to the client use DDESNA. This command will advise the client that data is ready. The client will then request the data. This request will be accepted through (DDEGTR or DDEGET.)

Example

```
++INCLUDE DDE.MAP

      IF    dhd'flg = dde’fsend                                  &
            PRINT "Sending data"                                 : &
            dde’rtn’string    = "The Data"                       : &
            XCALL DDESNR, dde, dde’scalc, dde’rtn’string, dde’status
```

See Also

DDEGTR, DDEGET

Synonym

dde’return’string(basic), dde’return’unf(basic), DDESNR(assembler)

DDESVA.SBR (xcall)

Save Advise Information. This routine is used in processing a hot link (or "Advise Start") request. To it you give an address where AlphaDDE can save information to identify the requesting client. This address is returned in DDESNA, below, when you have updated information and want to inform the client.

Libraries

`++INCLUDE DDE.MAP` *(optional)*

Prerequisite

Connection to AlphaDDE

Incoming message with “Start Advise” message type (3)

Syntax

```
XCALL DDESVA dde, dhl'link
```

`dde`	Is an AlphaBASIC structure defined in the appendix (this structure is defined in DDE.MAP).
`dhl'link`	Advise Link structure as defined in the appendix (this structure is defined in DDE.MAP as an element of an array of Advise Links.)

Input

None

Output

`dhl'link`	Advise Link structure.
hhd'flg	Message flags
hhd'fmt	Data format requested
hhd'bsd	Identifier of PC for the Response message
hhd'soc	Identifier of window for Response message
dhl'service	Service name of the message received
dhl'topic	Topic name of the message received
dhl'item	Item name of the message received

Comments

When creating a hot topic server any number of clients can request information simultaneously. For each request you must save the data block returned by **DDESVA** in some form of array. When the data updates the application will go through this array and inform the client (via **DDESNA**) that the data has changed.

After a **DDEPRC**, *DDM.FLG(*datbuf*)* will contain a three (3). This informs the server application to do the **DDESVA** to store off the client information for later **DDESNA** statements.

As clients drop off (DDM.*FLG(*datbuf*)* = 4), the application must remove the entry from the array. The *DDM.BSD(*datbuf*)* and *DDM.SOC(*datbuf*)* uniquely identify an entry in the list.

Example

```
++INCLUDE DDE.MAP

dde'add'hotlnk:
      IF    dhd'flg # dde'fswrm                          &
            RETURN

      FOR   dde'xxx = 1 TO dde'maxhot
       IF   dhl'inuse(dde'xxx) = 0                        &
            dhl'inuse(dde'xxx)      = -1                  : &
            XCALL       DDESVA, dde, dhl'link(dde'xxx)    : &
            dde'hotcnt  = dde'hotcnt + 1                  : &
            dde'xxx     = 99999

      NEXT  dde'xxx

      IF    dde'xxx < 99999                              &
            XCALL DDESNR, dde, dde'sadrej

      RETURN
```

See Also

DDESNA

Synonym

dde’add’hotlnk(basic), DDESVA(assembler)

DDECON (assembler)

Connect to AlphaDDE subsystem. This routine takes the specified input parameters and verifies their validity or sets up actual values if defaults were requested. It also locates the current system's AlphaNET address and the current job's socket number and saves these. Further, it opens a message socket to be used for ITC messages to and from SOCKS. It also registers the Topic Server with the AlphaNET Name Registry, builds the Topic Logon message, and sends the Logon message to SOCKS.

Libraries

```
search      netsym
copy        ddedef
copy        ddemem
```

Prerequisite

None

Syntax

```
DDECON ddeblk{, status}
```

`ddeblk` Is the AlphaDDE memory space pointer. In that memory area are a number of parameters that specify information about this Topic. (effective address)

`status` Return status of the call. Default is D7. (lword)

Input

DDM.SERVN is the address of the Service name to be used (ASCIZ), or 0 to use the default of "AlphaDDE". *ONLY "AlphaDDE" IS SUPPORTED AT THIS TIME*; use 0 for this parameter.

DDM.TOPICN is the address of the Topic name to be used (ASCIZ).

DDM.FORMATS is the address of a list of data formats that this Topic Server supports (see DDE.FMT in DDEDEF.M68). A value of 0 will support only the default format of CF_TEXT (ASCIZ). A maximum of 10 data formats can be supported per Item. ONLY CF_TEXT (ASCIZ) IS SUPPORTED AT THIS TIME; use 0 for this parameter.

DDM.CMDSUP is a bit mask of commands that this Topic Server can process (others will be rejected). 0 IS INVALID (implies no commands are supported)

DDM.MSGWAIT is the amount of time (in 10ms increments) that DDEGET (see below) will wait for a message. 0 = wait forever for message.

DDM.DISPLAY are flags that can be set to output different types of informative messages on the terminal. These are:

DDD$INFO Displays an ASCII message for any errors, and outputs messages indicating actions that occur within the Topic Server

DDD$VERB Is more Verbose than DDD$INFO (displays more information)

DDM.SYSTOP is the address of a routine to process System Topic requests for a Service other than AlphaDDE. NOT YET SUPPORTED; use 0 for this parameter.

DDM.ITEMLIST is the address of a string containing the names of Items supported by this Topic Server, if such can be enumerated. 0 indicates no list is available. This list is used by the AlphaDDE 'System' Topic. NOT YET SUPPORTED; MUST BE 0.

The **bolded** variables above signify required arguments. All others are optional.

Output

`status` Status of the call. 0 is no error. See appendix for additional error codes.

DDM.SERVN Service name, ASCIZ ("AlphaDDE" ONLY FOR NOW)

DDM.FORMATS List of WORDs of data formats supported (ONLY CF_TEXT = ASCIZ SUPPORTED NOW). Maximum number of data formats = 10.

DDM.MYADDR	current system's CPU ID
DDM.MYSOCK	current job's socket number
DDM.SOCKSA	CPU ID of SOCKS (if found via Name Registry)
DDM.SOCKSS	Job socket number of SOCKS (if found in Name Registry)
DDM.TOPBUF	Topic Logon message

Comments

This call *must* be made upon initialization of the program and prior to any other AlphaDDE calls. Call this routine only once within the bounds of a single program. Once communications are established you can perform any of the AlphaDDE commands. Upon termination from the program you should call DDEDIS to inform AlphaDDE that this service is no longer available.

Example

```
            search      sys
            search      syssym
            search      netsym
            copy        ddedef
            copy        ddemem

            autoextern              ; generate externs for library routines

            CMDS =      DDE$CRQST   ; Commands supported here (Request only)

      .ofini
      .ofdef      DDEADD, DDESERVERSIZE
      .ofsiz      IMPSIZ

            GETIMP      IMPSIZ,a4   ; get memory for DDE

      ; set up parameters for Connect (most parameters are 0 defaults)
            lea   a5,DDEADD(a4)
            lea   a1,TOPICNAME
            lea   a2,DDM.TOPICN(a5) ; pass Topic Name
      20$:  movb  (a1)+,(a2)+
            bne   20$
            movw  #CMDS,DDM.CMDSUP(a5)    ; Commands supported for Topic

            mov   #DDD$INFO,DDM.DISPLAY(a5)     ; Optional—displays info msgs

            DDECON      DDEADD(a4)  ; Connect to AlphaDDE subsystem
            jne   quit              ; error

      ; *** Begin Message Processing Loop
      rcvit:      ctrlc quit
            DDEGTR      DDEADD(a4)        ; Wait for an Item Request
            bmi   quit              ; Ctrl/C
            bne   error             ; Some terrible thing...
               \    /
               /    \

      TOPICNAME:  asciz /Simple/

      ITEMNM:           asciz /Alphabet/
                        even
```

See Also

DDEDIS

Synonym

dde’connect(basic), DDECON.SBR (xcall)

DDEDIS (assembler)

Disconnect from AlphaDDE subsystem. This routine is usually called after a CTRL/C or other termination. It sends a Topic Logoff message to SOCKS, removes itself from the Name Registry, and does other cleanup as necessary.

Libraries

```
search      netsym
copy        ddedef
copy        ddemem
```

Prerequisite

DDECON

Syntax

```
DDEDIS ddeblk
```

`ddeblk` Is the AlphaDDE memory space pointer. (effective address)

Input

None

Output

None

Comments

This call *must* be made upon termination of the program. Call this routine only once within the bounds of a single program. This call informs AlphaDDE that this service is no longer available.

Example

```
DDEDIS      DDEADD(a4)  ; Disconnect from the AlphaDDE subsystem
```

See Also

DDECON

Synonym

dde’disconnect(basic), DDEDIS.SBR (xcall)

DDEGET (assembler)

This routine checks for a pending ITC message, and waits for up to MSGWAIT for one. If a message is found, it is checked to see if it is an AlphaDDE message.

Libraries

```
search      netsym
copy        ddedef
copy        ddemem
```

Prerequisite

Connection to AlphaDDE

Syntax

```
DDEGET ddeblk{, status}
```

`ddeblk`	Is the AlphaDDE memory space pointer. (effective address)
`status`	Return status of the call. Default is D7. (lword)

Input

`datbuf`	Points to the base of the data buffer to be sent back to the client program.

Output

`status`	0 if an AlphaDDE message was received. -1 if Ctrl/C occurred. See appendix for additional codes.
DDM.DATAP	Address of data (if any) in received message
DDM.SOCKSA	CPU ID of SOCKS (if SOCKS Logon received)
DDM.SOCKSS	Job socket of SOCKS (if SOCKS Logon received)
DDM.HDRS	ITC & DDE headers of message received
DDM.COMMAND	Transaction request
DDM.FORMAT	Data format requested
DDM.PC	Identifier of PC for the Response message
DDM.WINDOW	Identifier of window for Response message
DDM.SERVICE	Service name of the message received
DDM.TOPIC	Topic name of the message received
DDM.ITEM	Item name of the message received

Comments

This command will wait up to a maximum of the time out period defined in *DDM.MSGWAIT*. Control will be returned to the calling program immediately following an incoming message or the time our period.

This function is most useful in Hot Link servers where the application program must wake up occasionally to check for a change in the data being requested.

To filter unwanted messages call **DDEPRC** after determining that a something other than a time out occurred.

DDEGET and **DDEPRC** are combined in the higher level **DDEGTR** function. Although **DDEGTR** does not offer the flexibility of the discreet functions, implementation is much simpler.

Example

```
rcvit:
      ctrlc       quit
      call        ChkHot            ; Check if time to send linked data

      DDEGET      @a5               ; Get a message
      bmi         quit              ; Ctrl/C
      beq         20$               ; Received a DDE message
                                    ; Got a non-DDE ITC message
      br    rcvit                   ; we ignore these...

20$:  DDEPRC      @a5               ; Verify message
      bne         rcvit             ; not Item Request, or some problem

      call  ProcessRequest          ; process an item here
      jmp   rcvit                   ; look for another
```

See Also

DDEPRC, DDEGTR

Synonym

dde’get’request(basic), DDEGET.SBR (xcall)

DDEGTR (assembler)

This routine waits for a pending ITC message. When one is received, it is checked to see if it is a properly formatted AlphaDDE message. If the message passes all checks, the Item Request command and data format are returned (in DDM.COMMAND and DDM.FORMAT in DDEMEM.M68), and the Item itself is returned in DDM.ITEM.

Libraries

```
search      netsym
copy        ddedef
copy        ddemem
```

Prerequisite

DDECON

Syntax

```
DDEGTR ddeblk{, item{, status}}
```

`ddeblk`	Is the AlphaDDE memory space pointer. (effective address)
`item`	Return address for the item. Default is A2. (effective address)
`status`	Return status of the call. Default is D7. (lword)

Input

None

Output

`item`	Points to the base of the returned item name string. This string is null terminated.
`status`	0 if an AlphaDDE message was received. -1 if Ctrl/C occurred. See appendix for additional codes.
DDM.DATAP	Address of data (if any) in received message
DDM.SOCKSA	CPU ID of SOCKS (if SOCKS Logon received)
DDM.SOCKSS	Job socket of SOCKS (if SOCKS Logon received)
DDM.HDRS	ITC & DDE headers of message received
DDM.COMMAND	Transaction request
DDM.FORMAT	Data format requested
DDM.PC	Identifier of PC for the Response message
DDM.WINDOW	Identifier of window for Response message
DDM.SERVICE	Service name of the message received
DDM.TOPIC	Topic name of the message received
DDM.ITEM	Item name of the message received

Comments

AlphaDDE "control" messages such as SOCKS Logon or Logoff are processed and returned. Otherwise, if the command ("Cold" Request, Start Hot Link, Poke, etc.) is not supported by this Topic Server, based on the CMDSUP parameter to DDECON, an error message is returned. The message is verified to be properly formatted and that the Service (MUST BE AlphaDDE AT PRESENT) and Topic are correct. It also checks that the data format requested is supported by this Topic Server. If an AlphaDDE message fails any check, an Item Response message is sent with the appropriate error code. If a message is received that is not in AlphaDDE format, it is ignored. If you need more control than this, see DDEGET.

Example

```
LOOP: DDEGTR      DDEADR(A4)              ; Get DDE Request
      BNE         QUIT                    ; Ctrl/C or horrible error
process request - check item, set up data to be returned
      DDESNR      DDEADR(A4),DATABF(A4)   ; send response
      BR          LOOP        ; do more
                              ; got DDE Item Request
```

See Also

DDESNR, DDEGET, DDEPRC

Synonym

dde’wait’request(basic), DDEGTR.SBR (xcall)

DDEPRC (assembler)

This routine checks to see if the requested command ("Cold Request", Start Hot Link, Poke, etc.) is supported by this Topic Server, based on the CMDSUP parameter to DDECON. It also verifies that the message is in the proper format and that the Service (MUST BE AlphaDDE AT PRESENT) and Topic are correct. It also checks that the data format requested is supported by this Topic Server. If the message passes all such checks, the Item Request command and data format are returned. If a message fails any check, an Item Response message is sent with the appropriate error code.

Libraries

```
search      netsym
copy        ddedef
copy        ddemem
```

Prerequisite

Connection to AlphaDDE

Message as received from **DDEGET**

Syntax

```
DDEPRC ddeblk{, status}
```

`ddeblk`	Is the AlphaDDE memory space pointer. (effective address)
`status`	Return status of the call. Default is D7. (lword)

Input

`datbuf`	Points to the base of the data buffer to be sent back to the client program.

Output

`status`	0 if an AlphaDDE message was received. -1 if Ctrl/C occurred. See appendix for additional codes.
DDM.DATAP	Address of data (if any) in received message
DDM.SOCKSA	CPU ID of SOCKS (if SOCKS Logon received)
DDM.SOCKSS	Job socket of SOCKS (if SOCKS Logon received)
DDM.HDRS	ITC & DDE headers of message received
DDM.COMMAND	Transaction request
DDM.FORMAT	Data format requested
DDM.PC	Identifier of PC for the Response message
DDM.WINDOW	Identifier of window for Response message
DDM.SERVICE	Service name of the message received
DDM.TOPIC	Topic name of the message received
DDM.ITEM	Item name of the message received

Comments

DDEGET and **DDEPRC** are combined in the higher level **DDEGTR** function. Although **DDEGTR** does not offer the flexibility of the discreet functions, implementation is simpler.

Example

```
rcvit:
      ctrlc       quit
      call        ChkHot            ; Check if time to send linked data

      DDEGET      @a5               ; Get a message
      bmi         quit              ; Ctrl/C
      beq         20$               ; Received a DDE message
                                    ; Got a non-DDE ITC message
```

```
        br    rcvit                    ; we ignore these...

20$:    DDEPRC      @a5                ; Verify message
        bne         rcvit              ; not Item Request, or some problem

        call  ProcessRequest           ; process an item here
        jmp   rcvit                    ; look for another
```

See Also

DDEPRC, DDEGTR

Synonym

dde’process’request(basic), DDEPRC.SBR (xcall)

DDESNA (assembler)

Send Advise Data Ready Message. After a hot link (Advise loop) has been initiated, you call this routine to indicate that there is new data. The client will issue a request for that data when he is ready.

Libraries

```
search      netsym
copy        ddedef
copy        ddemem
```

Prerequisite

Connection to AlphaDDE

Link data saved via DDESVA

Syntax

```
DDESNA ddeblk, savpnt{, status}
```

`ddeblk`	Is the AlphaDDE memory space pointer. (effective address)
`savpnt`	Is the address of the stored client info (effective address)
`status`	Return status of the call. Default is D7. (lword)

Input

`datbuf`	Points to the base of the data buffer to be sent back to the client program.
`savpnt`	Base pointer of a structure of DDE$ADVSZ bytes. The internal layout of this structure is defined in the appendix. This structure *must* have been previously loaded using the **DDESVA** function (initiated by an Advise Request.)

Output

`status`	0 if all ok. See appendix for additional codes.

Comments

When creating a hot topic server any number of clients can request information simultaneously. For each request you must save the data block returned by **DDESVA** in some form of array. When the data updates the application will go through this array and inform the client (via **DDESNA**) that the data has changed.

Example

```
; Check for linked data to send
ChkHot:
      tstb        LINKED(a4)        ; is the link active?
      beq         90$               ; no
      inc         COUNT(a4)         ; our timer...
      cmp         COUNT(a4),#10.
      blt         90$               ; not time yet
      clr         COUNT(a4)
      movb        DATABF(a4),d0
      inc         d0
      cmpb        d0,#03A
      bne         20$
      movb        #030,d0
20$:  movb        d0,DATABF(a4)
      TYPECR      Advising...
      DDESNA      @a5,HOTBUF(a4)
90$:  rtn
```

See Also

DDESVA

Synonym

dde’advise’all(basic), DDESNA.SBR (xcall)

DDESNR (assembler)

This routine sends a DDE Item Response message to SOCKS based on the size/flags field. If the size field is greater than or equal to 0, the data buffer parameter points to data to be returned to a PC window. If the size field is less than 0, it is a flag value indicating what non-data response is to be sent.

Libraries

```
search      netsym
copy        ddedef
copy        ddemem
```

Prerequisite

Connection to AlphaDDE

Received message of a client requesting data (via DDEGET or DDEGTR.)

Syntax

```
DDESNR ddeblk, datbuf, size{, status}
```

`ddeblk`	Is the AlphaDDE memory space pointer. (effective address)
`datbuf`	Address of send data buffer (effective address)
`size`	Size of send data buffer (lword)
`status`	Return status of the call. Default is D7. (lword)

Input

`datbuf`	Points to the base of the data buffer to be sent back to the client program.
`size`	The size parameter defines the size of the data buffer or the response type. *size* is either the absolute data buffer size (for binary or text data) or one of the following:
DDS$ASCIZ(0)	if the data is ASCIZ (library to calculate size)
DDS$ACKPOKE(-1)	if a Poke is to be acknowledged
DDS$NOITEM(-2)	if an Item not found message is to be sent
DDS$ERROR(-3)	if DDM.STAT and DDM.FLG contain status information to return
DDS$ACKADV(-4)	if an Advise Request (Start or Stop) is to be acknowledged. Note: Advise Start acknowledge is sent by DDESVA
DDS$REJADV(-5)	if an Advise Request is to be rejected
DDM.SERVICE	Service name for the Response message
DDM.TOPIC	Topic name for the Response message
DDM.ITEM	Item name for the Response message
DDM.PC	Identifier of PC for the Response message
DDM.WINDOW	Identifier of window for Response message

Output

`status`	0 if ok. See appendix for additional codes.

Comments

This command allows the application program to return data and information back to the client. This command should only be used after an item request (DDEGTR or DDEGET) has been received from the client. At that point the client has requested data and is ready for a response.

In the case of a hot link, when the application determines that the data has been updated and needs to be sent to the client use DDESNA. This command will advise the client that data is ready. The client will then request the data. This request will be accepted through (DDEGTR or DDEGET.)

Example

```
rcvit:
      ctrlc       quit
      DDEGTR      DDEADD(a4)        ; Wait for an Item Request
      bmi         quit              ; Ctrl/C
      bne         error             ; Some terrible thing...
; Received a DDE message
      lea         a5,DDEADD(a4)
      CMPSTR      ITEMNM, DDM.ITEM(a5)    ; Correct Item?
      beq         200$              ; Yes, process it          *
      typecr      <Unknown Item>
      mov         #DDS$NOITEM,d7    ; not my Item
      br          800$              ; return error (unknown item)
200$: mov         #DDS$ASCIZ,d7     ; ASCIZ, have function calc size
800$: DDESNR      DDEADD(a4),DATABF(a4),d7; Send Response
      beq         900$
      typecr      Error sending Response
900$: jmp         rcvit             ; look for another
```

See Also

DDEGTR, DDEGET

Synonym

dde’return’string(basic), dde’return’unf(basic), DDESNR.SBR (xcall)

DDESVA (assembler)

Save Advise Information. This routine is used in processing a hot link (or "Advise Start") request. To it you give an address where AlphaDDE can save information to identify the requesting client. This address is returned in DDESNA, below, when you have updated information and want to inform the client. Also sends an acknowledgment to the client confirming the Start Advise.

Libraries

```
search      netsym
copy        ddedef
copy        ddemem
```

Prerequisite

Connection to AlphaDDE

Message as received from **DDEGET**

Syntax

```
DDESVA ddeblk, savpnt{, status}
```

`ddeblk`	Is the AlphaDDE memory space pointer. (effective address)
`savpnt`	Is the address to store client info (effective address)
`status`	Return status of the call. Default is D7. (lword)

Input

`datbuf`	Points to the base of the data buffer to be sent back to the client program.
`savpnt`	Base pointer of a structure of DDE$ADVSZ bytes. The internal layout of this structure is defined in the appendix.

Output

`status`	0 if all ok. See appendix for additional codes.
`@savpnt`	Data structure for send advise.

Comments

When creating a hot topic server any number of clients can request information simultaneously. For each request you must save the data block returned by **DDESVA** in some form of array. When the data updates the application will go through this array and inform the client (via **DDESNA**) that the data has changed.

After a **DDEPRC**, *DDM.FLG(*datbuf*)* will contain a three (3). This informs the server application to do the **DDESVA** to store off the client information for later **DDESNA** statements.

As clients drop off (DDM.*FLG(*datbuf*)* = 4), the application must remove the entry from the array. The *DDM.BSD(*datbuf*)* and *DDM.SOC(*datbuf*)* uniquely identify an entry in the list.

Example

```
; local memory variables
.ofini
.ofdef      DATABF,     1024.
.ofdef      HOTBUF, DDE$ADVSZ
.ofdef      COUNT,      4
.ofdef      LINKED, 1
.ofsiz      IMPSIZ
      GETIMP      IMPSIZ,a4         ; get memory for local variables
      LEA         a5, DATABF(a4)
            \    /
            /    \

ProcessRequest:
      CMPSTR      ITEMNM,DDM.ITEM(a5)     ; Correct Item?
      beq         100$              ; Yes, process it
      typecr      Unknown Item
      mov         #DDS$NOITEM,d7          ; not my Item
```

```
        br          800$                    ; return error (unknown item)

100$:   movb        DDM.FLG(a5),d0          ; Get the request type
        cmpb        d0,#1                   ; Poke?
        bne         200$                    ; no
; We accept Pokes but don't process the data
        mov         #DDS$ACKPOKE,d7         ; Poke data ok
        jmp         800$

; Check if it's an Item I know
200$:   cmpb        d0,#0                   ; Data Request?
        bne         300$                    ; no
        clr         d7                      ; data is ASCIZ, return it
        br          800$

300$:   cmpb        d0,#3                   ; Start Advise?
        bne         400$                    ; no
        tstb        LINKED(a4)
        bne         480$                    ; already am linked
        DDESVA      @a5,HOTBUF(a4)          ; Save Link-ID
        incb        LINKED(a4)              ; tell myself I've got a link
        br          900$

400$:   cmpb        d0,#4                   ; End Advise?
        bne         500$                    ; no
        tstb        LINKED(a4)
        beq         480$                    ; Can't end what hasn't begun
        clrb        LINKED(a4)
        clr         COUNT(a4)

        mov         #DDS$ACKADV,d7          ; Request accepted
        br          800$

480$:   movw        #DDE$NOADV,DDM.STA(a5)  ; Refuse the advise request
        br          700$                    ; Advise already going

500$:   movw        #DDE$FMTER,DDM.STA(a5)  ; don't support other
                                            ; transaction types
        br          700$

; Send Error Response
700$:   movb        #4,DDM.FLG(a5)          ; Error
750$:   mov         #DDS$ERROR,d7
; Send Response Message
800$:   DDESNR      @a5,DATABF(a4),d7 ; Send Response
        beq         900$
        typecr      Error sending Response
900$:   rtn
```

See Also

DDESNA

Synonym

dde'add'hotlnk(basic), DDESVA.SBR (xcall)

Chapter Six
Programming and Integration Methods

The AlphaDDE libraries contain functions to make it easier to write AlphaDDE applications. This primarily means Topic Server programs.

Limitation: C and AcuCobol language functions are not yet available (only AlphaBASIC and M68)

M68 Assembly Language Techniques

Assembly language programs require COPYing DDEDEF.M68 and DDEMEM.M68. After impure space is acquired, all subsequent routines require a pointer to this memory. Topic Server program should include AUTOEXTERN so that the AlphaDDE macros can find their respective functions.

Routines are available in various flavors (i.e. ones that are easier to use but provide less flexibility, and ones that require more work but give maximum flexibility.

A sample using the easy method (as found in SMPTOP.M68)

```
; do initialization as needed (get memory, open files, etc.)
; Setup Connect parameters (Topic Name, options)
      DDECON  DDEADR(A4)
; Initialize and connect to AlphaDDE
LOOP: DDEGTR      DDEADR(A4)              ; Get DDE Request
      BNE         QUIT                    ; Ctrl/C or horrible error
; process request – check item, set up data (if any) to be returned
      DDESNR      DDEADR(A4), DATABF(A4)  ; send response
      BR          LOOP                    ; do more
QUIT: DDEDIS      DDEADR(A4)
                  EXIT
```

A sample using the a more flexible method (as found in HOTTOP.M68):

```
; do initialization as needed (get memory, open files, etc.)
; set up Connect parameters (Topic Name, options)
      DDECON      @A5         ; Connect to AlphaDDE
LOOP: DDEGET      @A5         ; Get ITC message
      BMI         QUIT        ; Ctrl/C
      BNE         ITC         ; Nothing we want, timeout
      DDEPRC      @A5         ; Check DDE type
      BNE         LOOP        ; Nothing we want
      check request type
            -- process Request
      Get Requested Data for Response
            -- process Poke
      Process Received Data
            -- process Hot link start
      DDESVA      @A5, @A2    ; save client identification and Item
      Set Link Active
            -- process Hot link end
      Clear Link Active
      DDESNR      @A5,@A2     ; send response
      BR          LOOP        ; do more
ITC:  ITC message?
      BNE         LINKS       ; no
      Process other ITC message as desired, if at all
      BR          LOOP
LINKS:Link active?
      BNE         LOOP        ; no
      check for host link data to send
      BNE         LOOP
      DDESNA      @A5, @A2    ; if so (Send "Advise Data Ready" to host)
      BR          LOOP
QUIT: DDEDIS      @A5
      EXIT
```

Chapter Seven
Sample Topic Servers and Windows DDE Programs

Sample Programs

A number of sample programs are provided in both AlphaBASIC and Assembly language (with C and AcuCobol to follow in a future release) for the Alpha, and Visual Basic or C for the PC. Some of the programs for the two systems are written as pairs to create simple applications, while others are independent. For example, the Visual Basic program ISAMDDE is designed to access the Topic Server ISMTOP.LIT.

You will see that it is very easy to use Microsoft Visual Basic to access AlphaDDE Topic Servers. The sample programs will show this. As another reference, you may wish to use the January 1995 issue of the *Visual Basic Programmer's Journal* , which has an excellent article on using DDE with Visual Basic.

The sample programs provided for the two systems (Alpha Micro and PC) are:

Sample AMOS Topic Servers (in DDE:, [7,55])

AlphaBASIC

SMPTOP.BAS	Very simple Request/Response Topic Server using high level AlphaBASIC library.
HOTTOP.BAS	Topic Server that supports Advise ("Hot") links. Supports multiple client windows, advises all when its reference data changes.
TOPSER.BAS	Simple Topic Server using lower level AlphaBASIC XCALL's.

AMOS Assembly

DATSER.LIT	Topic Server that returns file data formatted according to file extension. Used with two of the sample Visual Basic programs (DISPFILE and GRAPHER). A flag may be defined when assembling DATSER to restrict access to PPNs. This method requires that a file named PUBLIC.PPN exist in the PPN for DATSER to allow access to those files. Requires about 25k.
HOTTOP.LIT	Topic Server that supports Advise ("Hot") links. Works with the Visual Basic sample program HOTLINK. This simple version only works with one client at a time. Requires about 15k.
ISMTOP.LIT	Topic Server that accesses an ISAMP file (a parts database). Retrieves parts data either by key or sequentially if special key values are supplied. Works with the Visual Basic sample program ISAMDDE, and used for Lotus 123 sample (see next chapter). Requires about 20k.
MEDTOP.LIT	Topic Server that uses the lower level calls (DDEGET and) rather than the less flexible DDEGTR used in SMPTOP. Requires about 15k.
SMPTOP.LIT	Very simple Request/Response Topic Server. Used with the sample C programs (including the initial installation verification). Also used in the Visual Basic DDE Experimenter example. Requires about 15k.

Sample PC Code

Visual Basic (directory ALPHADDE\VB)

DDE	Experimenter to allow access to any DDE service. You specify the Service, Topic, and Item, then Connect and issue Requests, Pokes, or Executes.

DISPFILE	Use Topic Server DATSER.LIT to display data from a file (the data displayed is what will fit in the display window).
GRAPHER	Use Topic Server DATSER.LIT to build a graph from a AlphaCALC Cell-ID file.
HOTLINK	Use Topic Server HOTTOP.LIT to demonstrate a simple Advise (Hot) link. This simple program returns a count that is updated every few seconds.
ISAMDDE	Use Topic Server ISMTOP.LIT to access an ISAMP Parts file. You can access specific keys, or move sequentially forward or backwards in the file by using special key values.

Example Use of the Visual Basic DDE Experimenter

Start the DDE Experimenter. You need to have SMPTOP.LIT running on the AMOS server. Enter *AlphaDDE* as the Application (Service), *Simple* as the Topic, and *Alphabet* as the Item. Select *Manual* and then *Connect*. The text box "Destination Data" should display several numbered lines of ASCII data in sequence (0 !"#$%'()*+,-./0123456789, etc.).

This test uses the DDE Experimenter program (one that lets you enter any Service, Topic, and Item) to make a DDE request. The request is passed through the AlphaDDE subsystem to the Topic Server SMPTOP.LIT, which responds.

Microsoft or Borland C for Windows (directory ALPHADDE\C)

ddeclnt	Access Topic Server SMPTOP.LIT to demonstrate simple C code. This provides menu items to Connect, Disconnect, Request, and Poke.
ddetest	Access Topic Server SMPTOP.LIT to verify installation.

Chapter Eight
Using DDE in Windows Applications

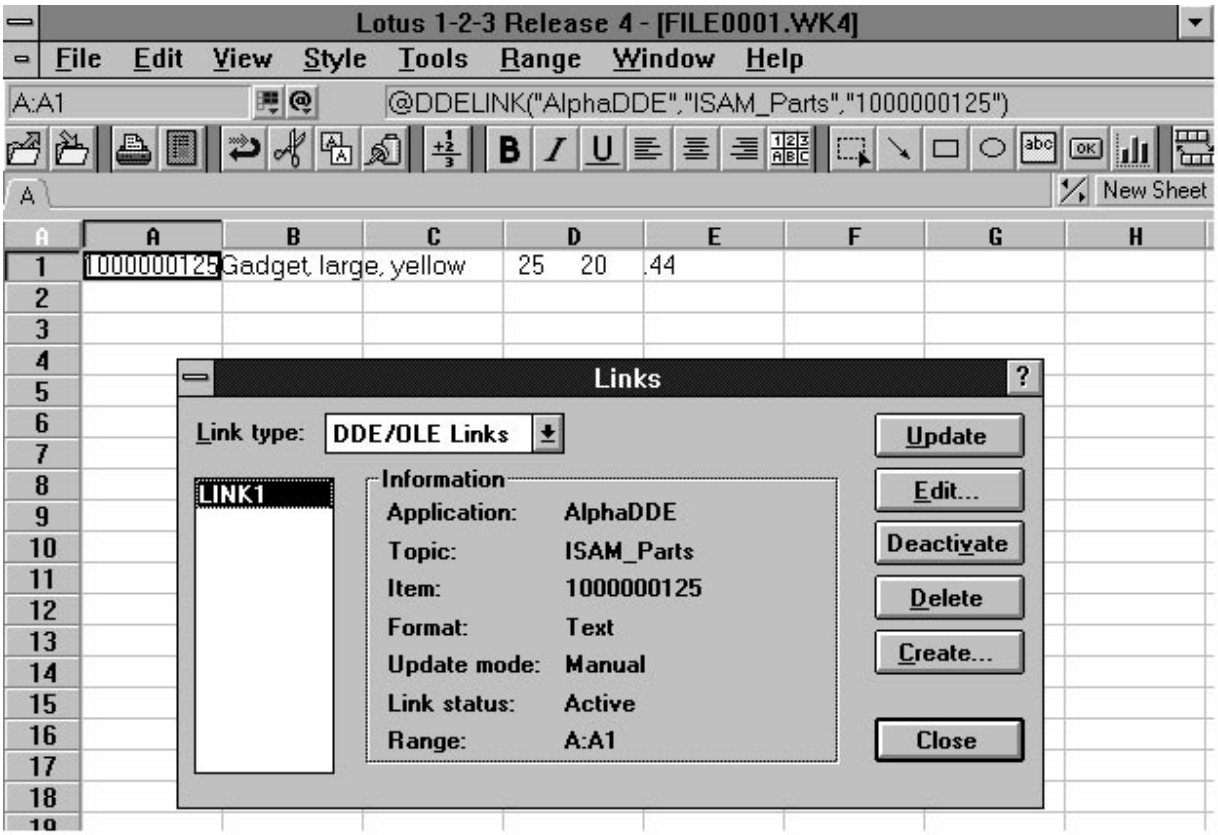
Sample Application Usage

This chapter provides some examples of how to use standard Windows applications with AlphaDDE. This section should expand greatly with future releases of the product.

Lotus 123

Lotus 123 allows you to easily access AlphaDDE by using the *Links* dialogue box under the *Edit* menu. This dialogue box allows you to specify the Application (*AlphaDDE*), Topic (your *Topic Server's registered name*), and the *Item* you wish to access. The Update mode for a simple request is *Manual*. You can of course use Lotus macros to store this type of request, or to embed it as a function in Lotus 123.

Lotus 123 Links Dialogue Box



Microsoft Access

You can set up macros in Microsoft Access to access AlphaDDE resources. Here is a sample macro that accesses the AlphaDDE sample Topic Server *ISAM_PARTS*, and formats the record data received:

```
Sub MAIN
      Dim chan, text$, LastItem$, ItemCount
      Dim ItemNumber, NoMoreData

      chan = DDEInitiate (“AlphaDDE”, “ISAM_Parts”)
      On Error Goto ErrorHandler
      NoMoreData = false
      LastItem$ = “”
      text$ = DDERequest$ (chan, “1000000001”)
      text$ = Left$ (text$, 10) + “ “ + Right$(text$, Len(text$) - 10) +
Chr(13) + Chr(10)
      Font “Courier New”
      Insert text$
      ItemCount = 0
      While Not NoMoreData
            text$ = DDERequest$ (chan, “1000000001”)
            text$ = Left$ (text$, 10) + “ “ + Right$(text$, Len(text$) -
10) + Chr(13) + Chr(10)
            If text$ = LastItem$ Then
                  NoMoreData = true
            Else
                  Font “Courier New”
                  Insert text$
                  LastItem$ = text$
                  ItemCount = ItemCount + 1
            Endif
      Wend
      Goto EndofMain

ErrorHandler:

EndofMain:
End Sub
```

Microsoft Word

You can write Word Basic macros to access AlphaDDE similar to how you do so in Access. The only real differences are in the language syntax differences (just because they’re both Microsoft products doesn’t mean that they use the same language).

Chapter Nine
Error Handling

Introduction

AlphaDDE contains many components -- DDE clients on PCs, our DDE server (AlphaDDE.exe) on those PCs, the communications link including TCP/IP on both systems, SOCKS.LIT on the Alpha, and Topic Servers. Different type of failures may (and will) occur on each of these components., This section attempts to address the types of problems that may occur, how each component should handle those problems, and other error situations.

Problems to Address

- Any component may crash while a link is active
- A user may abort any program
- The DDE Client program may attempt to do things that AlphaDDE.exe does not support
- A Topic Server may attempt to do things that the libraries do not support
- A request may timeout, due to link errors, program aborts, or insufficient time allowed for response
- Either system (PC or host) may crash or be rebooted

Problem Handling

Topic Server Code Requirements

Condition: User aborts program

Action: Must do Disconnect (to force Topic logoff)

DDE Client Programs

Condition: Timeout occurred while using cold link

Action: (Depending on application) either quit or attempt to reconnect

Condition: Timeout occurred while using hot link

Action: (Depending on application) either quit or attempt to reconnect

Condition: Cannot Connect

Action: (Depending on application) either quit or continue to attempt to connect

Condition: Disconnect received when using cold link

Action: (Depending on application) either quit or attempt to reconnect

Condition: Disconnect received when using hot link

Action: (Depending on application) either quit or attempt to reconnect (reconnect will require restarting hot link)

Condition: User aborts program

Action: Sends Disconnect to AlphaDDE.exe.

Topic Server Libraries

Condition: Socks logoff (or never logged on)
Action: flag status; do not send data until Socks Logon received
Condition: Socks logon
Action: flag status, send Topic Logon message
Condition: 'Advise Stop' message received
Action: close hot link
Condition: User aborts program
Action: send Topic Logoff to Socks

SOCKS.LIT

Condition: Connection lost
Action: remove PC from connections list
Condition: Topic Server logoff
Action: remove Topic from active list, pass Topic logoff to AlphaDDE.exe
Condition: Topic Server lost (future - not currently detected)
Action: remove Topic from active list, pass Topic logoff to AlphaDDE.exe
Condition: Response from Topic Server for unconnected PC
Action: return Advise Stop
Condition: User aborts program
Action: send Socks Logoff to all connected PCs and Topic Servers

AlphaDDE.exe

Condition: Connection lost
Action: (hot link) send Disconnect for any current transactions
Action: (cold link) ignore. Return error if client attempts another transaction before the connection is reestablished
Condition: Timeout during data request
Action: (hot link) send Disconnect
Action: (cold link) return error
Condition: Topic deleted
Action: (hot link) send Disconnect for any current transactions for Topic
(cold link) ignore. Next transaction will return error if Topic not restarted by then
Condition: Socks logoff
Action: (hot link) send Disconnect for any current transactions
(cold link) ignore. Next transaction will return error, if link not reestablished
Condition: Response from Topic Server for inactive conversation
Action: (hot link) send 'Conversation Unknown' message to Socks (translates to Advise Stop message back to Topic Server)
Condition: User aborts program
Action: send Disconnect to any active transaction. Send 'PC Logoff' to Socks

AMOS system rebooted (Reset or MONTST)

If the AMOS host is reset, the PC has no way to know that the connection is lost until TCP/IP times out, which takes a number of minutes. During that time, AlphaDDE.exe will show the same status (Logged on, Topics registered) as before. Any client attempts to access those Topics will result in a timeout. Even if the AMOS system reboots before the TCP/IP connection on the PC times out (which it usually will during a MONTST), the PC will not know to reconnect. In this situation, the AlphaDDE window will indicate that the connection is up, the AMOS host will be up, but there is no connection. To correct this, AlphaDDE.exe on the PC must be closed and restarted. This problem is expected to be fixed (by sending "keep-alive" messages) in the next version of AlphaDDE.

Appendix A
Data Structures

AlphaDDE Parameter Limits

AlphaDDE header size	32
Maximum Service name length	32*
Maximum Topic name length	28*
Maximum Item name length	256*
Maximum Execute string length	256* **
Maximum Data item length	1024***
Maximum AlphaDDE message size	1377

* including final NULL (ASCIZ format)

** passed as an Item string

*** may include additional final NULL if text (ASCIZ) format

DDE "System" Topic Items

A DDE System Topic returns information about a DDE Server such as Topics supported, data formats supported, etc. (see below). AlphaDDE will support the System Topic. Note: Lists are returned in CF_TEXT (ASCIZ, CR/LF line terminators) format, with TAB as separator.

Topics	All Topics supported at present
SysItems	Which of these Items is supported by this Server
Status	String equal to "Ready" or "Not Ready"
Formats	Clipboard formats application supports

AlphaBASIC

AlphaBASIC xcalls and high level routines need the data structures as defined below (These structures are defined in DDE.MAP.)

DDE Data Block Structure

```
MAP1  dde
 MAP2 dde'flags   , X   , 4   ! flags (for future use)
 MAP2 dde'myaddr  , X   , 4   ! My AlphaNET address
 MAP2 dde'mysock  , B   , 2   ! My socket number
 MAP2 dde'socksa  , X   , 4   ! AlphaNET address of SOCKS
 MAP2 dde'sockss  , B   , 2   ! Socket Number of SOCKS
 MAP2 dde'hdr                 ! AlphaDDE message header
  MAP3      dhd'id      , B   , 2   !
  MAP3      dhd'hsz     , B   , 1   !
  MAP3      dhd'msg     , B   , 1   ! Transaction Request
  MAP3      dhd'flg     , B   , 1   ! Message flags
  MAP3      dhd'res0    , X   , 1   !
  MAP3      dhd'fmt     , B   , 2   ! Data format requested
  MAP3      dhd'dsz     , B   , 2   ! Data size
  MAP3      dhd'stat    , B   , 2   ! Return status
  MAP3      dhd'add     , X   , 4   !
  MAP3      dhd'soc     , X   , 4   ! Identifier of requesting win
  MAP3      dhd'bsd     , X   , 4   ! Identifier of requesting PC
  MAP3      dhd'res1    , X   , 8

 MAP2 dde'service , S   , 32  ! Service Name requested
```

```
MAP2 dde'topic   , S   , 32  ! Topic Name requested
MAP2 dde'item    , S   , 256 ! Item Name requested
MAP2 dde'datap   , X   , 4   ! Address of data
MAP2 dde'msgwait , B   , 2   ! Number of ticks to wait for msg
MAP2 dde'display , X   , 4   ! Display (debugging) flags
MAP2 dde'maxlen  , B   , 2   ! Maximum data packet size
MAP2 dde'maxpen  , B   , 2   ! Maximum messages pending
MAP2 dde'opnblk  , X   , 10  ! OpenMessage parameter block
MAP2 dde'opnnam  , X   , 10  ! OpenMessage block for name reg.
MAP2 dde'rcvadd  , X   , 4   ! Receive buffer address
MAP2 dde'xmtadd  , X   , 4   ! Transmit buffer address
MAP2 dde'nambuf  , S   ,2000 ! Working space for Name Registry

! beginning of Server-only parameters
MAP2 dde'servn   , S   , 32  ! Service Name supported here
MAP2 dde'topicn  , S   , 32  ! Topic Name supported here
MAP2 dde'formats(10),B , 2   ! Data Formats supported (0=text only)
MAP2 dde'fmttrm  , X   , 2   ! Formats terminator
MAP2 dde'cmdsup  , B   , 2   ! Commands supported (1 byte used)
MAP2 dde'systop  , X   , 4   ! System Topic processing (s/b 0)
MAP2 dde'topbuf  , X   , 122 ! Topic Buffer
MAP2 dde'res     , X   , 500 ! bug somewhere...
MAP2 dde'xmtbuf  , X   ,1408 ! Transmit buffer (default)
MAP2 dde'rcvbuf  , X   ,1408 ! Receive buffer (default)
```

DDE Hot Link Data Structure

```
MAP3      dhl'hdr
 MAP4     hhd'id            , B   , 2   !
 MAP4     hhd'hsz           , B   , 1   !
 MAP4     hhd'msg           , B   , 1   ! Transaction Request
 MAP4     hhd'flg           , B   , 1   ! Message flags
 MAP4     hhd'res0          , X   , 1   !
 MAP4     hhd'fmt           , B   , 2   ! Data format requested
 MAP4     hhd'dsz           , B   , 2   ! Data size
 MAP4     hhd'stat          , B   , 2   ! Return status
 MAP4     hhd'add           , X   , 4   !
 MAP4     hhd'soc           , X   , 4   ! Identifier of requesting win
 MAP4     hhd'bsd           , X   , 4   ! Identifier of requesting PC
 MAP4     hhd'res1          , X   , 8
MAP3      dhl'service       , S   , 34  ! Service Name requested
MAP3      dhl'topic         , S   , 34  ! Topic Name requested
MAP3      dhl'item          , S   , 258 ! Item Name requested
```

Assembler

Assembly language programs using AlphaDDE are required to use the following structures for calls to the any of the AlphaDDE functions (these structures are defined in DDEMEM.M68.)

DDE Data Block Structure

```
.ofini
.ofdef      DDM.FLAGS,  4           ; flags (for future use)
.ofdef      DDM.MYADDR, 4           ; Topic Server Systems Alphanet Addr
.ofdef      DDM.MYSOCK, 2           ; Topic Server's stdeet #
.ofdef      DDM.SOCKSA, 4           ; AlphaNET address of SOCKS
.ofdef      DDM.SOCKSS, 2           ; Socket Number of SOCKS
;
.ofdef      DDM.HDR,    DDE$HSZ           ; AlphaDDE message header
      DDM.CMD     = DDM.HDR+DDE.MSG ; B Transaction request
      DDM.FLG     = DDM.HDR+DDE.FLG ; B Message flags
      DDM.FMT     = DDM.HDR+DDE.FMT ; W Data Format requested
      DDM.DSZ     = DDM.HDR+DDE.DSZ ; W Data size
      DDM.STA     = DDM.HDR+DDE.STA ; W Return status
      DDM.PC      = DDM.HDR+DDE.BSD ; L Identifier of requesting PC
      DDM.WIN     = DDM.HDR+DDE.SOC ; L Identifier of requesting window
.ofdef      DDM.SERVICE,      32.   ; Service Name requested
.ofdef      DDM.TOPIC,  32.         ; Topic Name requested
.ofdef      DDM.ITEM,   256.        ; Item Name requested
.ofdef      DDM.DATAP,  4           ; Address of data
```

```
;
.ofdef      DDM.MSGWAIT,      2     ; Number of ticks to wait for msg
.ofdef      DDM.DISPLAY,      4     ; Display (debugging) flags
      DDD$INFO    =     1           ; Informational messages
      DDD$VERB    =     2           ; Verbose message mode
.ofdef      DDM.MAXLEN, 2           ; Maximum data packet size
.ofdef      DDM.MAXPEN, 2           ; Maximum messages pending
.ofdef      DDM.OPNBLK, OM.SIZ      ; OpenMessage parameter block
.ofdef      DDM.OPNNAM, OM.SIZ      ; OpenMessage block for name reg.
.ofdef      DDM.RCVADD, 4           ; Receive buffer address
.ofdef      DDM.XMTADD, 4           ; Transmit buffer address
.ofdef      DDM.NAMBUF, 2000.       ; Working space for Name Registry
.ofsiz      DDECLIENTSIZE
; beginning of Server-only parameters
.ofdef      DDM.SERVN,  32.         ; Service Name supported here
.ofdef      DDM.TOPICN, 32.         ; Topic Name supported here
.ofdef      DDM.FORMATS,22.         ; Data Formats supported (0=text)
.ofdef      DDM.CMDSUP, 2           ; Commands supported (1 byte used)
      DDE$CRQST   = 1               ; Accept requests
      DDE$CPOKE   = 2               ; Accept poke data
      DDE$CADV    = 4               ; Accept advise requests
      DDE$CEXEC   = 8.              ; Accept Execute requests
      DDE$CALL    = DDE$CRQST!DDE$CPOKE!DDE$CADV!DDE$CEXEC
.ofdef      DDM.SYSTOP, 4           ; System Topic processing (s/b 0)
.ofdef      DDM.TOPBUF, MS.DAT+DDE$HSZ+DDE$SRVSZ+DDE$TOPSZ; Topic Buffer
.ofdef      DDM.INQBUF, MS.DAT+DDE$HSZ    ; Topic Inquiry Message
.ofdef      DDM.REGTOP, DDE$TOPSZ         ; Register Topic
.ofdef      DDM.RES,    500.-MS.DAT-DDE$HSZ-DDE$TOPSZ ; Reserved
.ofsiz      DDENOBUFSIZE
.ofdef      DDM.XMTBUF, MS.DAT+DDE$MAXSZ  ; Transmit buffer (default)
.ofdef      DDM.RCVBUF, MS.DAT+DDE$MAXSZ  ; Receive buffer (default)
.ofsiz      DDESERVERSIZE
```

DDE Hot Link Data Structure

```
DDE$ADVSZ   = DDE$HSZ+DDE$SRVSZ+DDE$TOPSZ+DDE$ITMSZ
```

AlphaDDE Message Header Format

The following is the message format in the data portion of the ITC message, starting at MS.DAT:

```
DDEID       word              ; AlphaDDE message identifier = "Dd"
DDEHSZ      byte              ; Header size (to allow for changes)
                              ; currently 32 bytes
DDEMSG      byte              ; Message Type
      DDM$SOCKS   = 1         ; SOCKS logon,logoff
      DDM$TOPSR   = 2         ; TOPSER logon, logoff
      DDM$RQST    = 3         ; Item Request
      DDM$RSPNS   = 4         ; Item Response
      DDM$ERROR   = 5         ; Response to Message Format errors
      DDM$DSTUNK  = 6         ; Destination Unknown
DDEFLG      byte              ; Flags
DDERES0     byte              ; reserved
DDEFMT      word              ; Data Format:  Windows Clipboard style...
; Limitation: at present, the only data format supported is a null
; terminated string (ASCIZ)
; Predefined Clipboard Formats from WINDOWS.H
      CF$TEXT     = 1         ; ASCIZ (CR/LF End-of-line)
      CF$BITMAP   = 2         ; Bitmap
      CF$METAF    = 3         ; Metafile Picture
      CF$SYLK     = 4         ;
      CF$DIF      = 5         ; DIF
      CF$TIFF     = 6         ; TIFF
      CF$OEMTX    = 7         ; DOS-font ASCIZ
      CF$DIB      = 8
      CF$PALETTE  = 9
      CF$PENDATA  = 10
      CF$RIFF     = 11
      CF$WAVE     = 12
DDEDSZ      word              ; Size of data field
```

```
DDESTA      word                ; Status
      DDE$NOERR   = 0           ; No Errors (ok-fine)
      DDE$NOITM   = 1           ; Item Unknown
      DDE$NOADV   = 2           ; Advise Not supported
      DDE$NOTPK   = 3           ; Topic cannot Poke
      DDE$NOIPK   = 4           ; Item cannot Poke
      DDE$NOTOP   = 5           ; Topic Unknown
      DDE$FMTER   = 6           ; Message format error
      DDE$NOSER   = 7           ; Service Unknown
      DDE$NORQST  = 8.          ; Topic cannot do Data requests
      DDE$ADVREJ  = 9.          ; Rejected Advise Request
      DDE$UNKPC   = 10.         ; Unknown PC
      DDE$UNKCNV  = 11.         ; Unknown Conversation
DDEADD      dword               ; PC's network address
DDESOC      dword               ; PC task's socket (DDE Client handle?)
DDEBSD      dword               ; BSD socket# (PC identification)
DDERES      byte[8]             ; reserved
DDESRV      asciz "S=ssss"      ; Service String, from DDE client
DDETOP      asciz "T=tttt"      ; Topic String, from DDE client or
                                ; Topic Server
DDEITM      asciz "I=iiii"      ; Item String, from DDE client
DDEDAT      asciz "D=dddd"      ; Data String, from Topic Server or
                                ; DDE Client
DDE$HSZ           = 32.         ;* Header Size
DDE$SRVSZ         = 3 + 31.     ; Maximum Service string (incl S= and
                                ; final NULL)
DDE$TOPSZ         = 3 + 31.     ; Max Topic string (incl T= and final
                                ; NULL)
DDE$ITMSZ         = 3 + 255.    ; Max Item string (incl I= and final NULL)
DDE$DATSZ         = 3 + 1024    ; Max Data string (incl D= and final NULL)
DDE$MAXSZ   = DDE$HSZ+DDE$SRVSZ+DDE$TOPSZ+DDE$ITMSZ+DDE$DATSZ+1
DDE$ADVSZ   = DDE$HSZ+DDE$SRVSZ+DDE$TOPSZ+DDE$ITMSZ   ; For Advise/Ack
```

Appendix B
Error Codes

AlphaDDE, ITC and Name Registry error codes are listed below.

```
          ERRDDEFUNC  = 32.                ; DDE Function return values
               ERRITC  = 64.                ; ITC Message Service errors
          ERRNAMEREG  = 96.                ; Name Registry errors

32          ERRNOTDDE  = ERRDDEFUNC        ; Received ITC message not DDE format
33          ERRBADHDR  = ERRDDEFUNC+1      ; DDE message not formatted properly
34          ERRNOCMDS  = ERRDDEFUNC+2      ; CMDSUP invalid; no cmds supported
35          ERRNOTOPIC = ERRDDEFUNC+3      ; No Topic Name supplied
36        ERRSERVNSUP  = ERRDDEFUNC+4      ; Services other than "AlphaDDE" n/s
37          ERRMSGPAR  = ERRDDEFUNC+5      ; MAXLEN,MAXPEN,RCVADD not set
38           ERRNOMSG  = ERRDDEFUNC+6      ; timeout, no message received
39           ERRNOMEM  = ERRDDEFUNC+7      ; no memory available
40           ERRNOSOCK = ERRDDEFUNC+8.     ; couldn't open message socket
41            ERRNOJOB = RRDDEFUNC+9.      ; couldn't find job socket (!?!?)
42           ERRNONET  = ERRDDEFUNC+10.    ; couldn't find AlphaNET network
43            ERRDONE  = ERRDDEFUNC+11.    ; no request; control proc. done
44         ERRNOSOCKS  = ERRDDEFUNC+12.    ; Socks not available (Can’t get into restautant)
45            ERRNOPC  = ERRDDEFUNC+13.    ; PC not available (but can still eat)
46          ERRDUPTOP  = ERRDDEFUNC+14.    ; Topic Name already in use

65              M$ENSK = ERRITC+1          ; No socket is open for the sending process
66              M$ENMP = ERRITC+2          ; No messages are pending
67              M$ENMS = ERRITC+3          ; O/S does not support message system
68              M$EDNN = ERRITC+4          ; Destination network does not exist
69              M$ENNN = ERRITC+5          ; Destination node does not exist
70              M$ESNN = ERRITC+6          ; Destination socket does not exist
71              M$ESAE = ERRITC+7          ; Socket already exists
72              M$ENMB = ERRITC+010        ; No message buffers are available
73              M$ENQB = ERRITC+011        ; No queue blocks are available
74              M$EAOP = ERRITC+012        ; Argument address outside caller's part
75              M$EDSF = ERRITC+013        ; Destination socket is full
76              M$EDSN = ERRITC+014        ; Destination socket not enabled
77              M$EMTL = ERRITC+015        ; Message length > destination maximum
78              M$EATO = ERRITC+016        ; Ack timeout

96            ERRNRTO  = ERRNAMEREG        ; Name registry timeout
97          ERRNRISREG = ERRNAMEREG+1      ; Name already registered
98           ERRNRFULL = ERRNAMEREG+2      ; Name Registry is full
99         ERRNRNOTREG = ERRNAMEREG+3      ; Name is not registered (not found)
100        ERRNRCORRUP = ERRNAMEREG+4      ; Name Registry structure corrupted
```

Appendix C
AlphaDDE Message Types

This appendix describes the format of the actual messages sent by AlphaDDE. This information is for reference only; the Topic Server library functions handle all message processing.

SOCKS Logon/Logoff

This message is sent from SOCKS to indicate its service status. Destinations are Topic Servers and AlphaDDE.exe. The On-Line message is used to request a DDE_MSG_TOPSER message from Topic Servers.

DDM$SOCKS = 1

DDEFLG	0	SOCKS is "on-line". Used for reconnection to Topic Servers
DDEFLG	1	SOCKS is shutting down
DDEFLG	2	SOCKS inquiry - Sent by Topic Server to check if SOCKS is running. Response is a SOCKS logon.
DDESTA		0
DDEADD, DDESOC		AlphaNET address of SOCKS

Required Fields

DDEID, DDEMSG, DDEFLG, DDEADD, DDESOC always required

Topic Logon/Logoff

This message is sent from Topic Servers to SOCKS and from SOCKS to AlphaDDE to indicate what Topic(s) is/are supported. Topic Servers need to be prepared to send this message in response to a DDE_MSG_SOCKS "On-Line" message.

DDM$TOPSR = 2

DDEFLG	0	Topic is available for DDE Clients to reference
DDEFLG	1	Topic Server is shutting down.
DDEFLG	2	Topic Logon Acknowledgement (SOCKS to Topic Server)
DDEFMT		Data format supported by this Topic (1=text) (each format supported requires a separate Topic message)
DDESTA		0
DDEADD, DDESOC		AlphaNET address of Topic Server
DDESRV		"S=ssss"-format string containing Service (always AlphaDDE for now)
DDETOP		"T=tttt"-format string containing Topic served

Required Fields

DDEID, DDEMSG, DDEFLG, DDEADD, DDESOC always required

DDESRV, DDETOP, DDEFMT

Item Request

This message is sent from SOCKS to a Topic Server to initiate a data request. The Topic Server is required to respond with a DDE_MSG_RSPNS.

DDM$RQST = 3

DDEFLG	0	Send Data for Specified Item (COLD Request)

DDEFLG	1	Here is Item String (POKE)
DDEFLG	2	Begin Hot Link Not Used -- At this level all Advises are Warm
DDEFLG	3	Begin Warm Link
DDEFLG	4	End Link (Warm or Hot)
DDEFLG	5	Execute

DDESTA	0
DDEADD, DDESOC	AlphaNET address of DDE Client
DDESRV	"S=ssss"-format string containing Service (always AlphaDDE for now)
DDETOP	"Topic=tttt"-format string containing Topic requested
DDEITM	"Item=iiii"-format string containing Item requested (or Execute string)

Required Fields

DDEID, DDEMSG, DDEFLG, DDEADD, DDESOC always required

DDESRV, DDETOP, DDEITM

Optional Fields

DDEDAT (contains Item Data for "POKE", DDEFLG=1)

Item Response

This message is sent from a Topic Server to SOCKS in response to a data request (DDE_MSG_RQST).

DDM$RSPNS = 4

DDEFLG	0	Sending Data for Specified Item
DDEFLG	1	Advise Request Accepted
DDEFLG	2	Advise Data Available
DDEFLG	3	Poke Data Accepted
DDEFLG	4	Error -- see DDESTA
DDESTA	0	No error
DDESTA	1	Item Unknown
DDESTA	2	Advise (Hot or Warm link) not supported for Topic
DDESTA	3	Poke not supported for Topic
DDESTA	4	Poke not supported for Item
DDESTA	5	Topic unknown
DDESTA	6	Message Format error
DDESTA	7	Service unknown
DDESTA	8	Topic cannot do request
DDESTA	9	Reject Advise Request
DDEADD, DDESOC		AlphaNET address of DDE Client
DDESRV		"S=ssss"-format string containing Service (always AlphaDDE for now)
DDETOP		"Topic=tttt"-format string containing Topic requested
DDEITM		"Item=iiii"-format string containing Item requested
DDEDAT		"Data=dddd"-format string containing Data requested

Required Fields

DDEID, DDEMSG, DDEFLG, DDEADD, DDESOC always required

DDESRV, DDETOP, DDEITM

Optional Fields

DDEDAT (contains Item Data for DDEFLG=0)

Message Format Error

This message is sent to the PC from a Topic Server or SOCKS in response to a message that is formatted so badly that the intended recipient cannot be determined:

```
DDM$ERROR    = 5
    DDEFLG     0
    DDESTA     6      Message Format Error
DDEADD, DDESOC   AlphaNET address of DDE Client
            DDESRV   "S=ssss"-format string containing Service (always AlphaDDE for now)
            DDETOP   "Topic=tttt"-format string containing Topic requested
            DDEITM   "Item=iiii"-format string containing Item requested
```

Required Fields

DDEID, DDEMSG, DDEFLG, DDEADD, DDESOC always required

DDESRV, DDETOP

Destination Unknown

This message is sent to SOCKS from the PC if a DDE Client program has been aborted:

```
DDM$DSTUNK = 6
    DDEFLG     0
    DDESTA    11      Unknown Conversation
DDEADD, DDESOC   AlphaNET address of DDE Client
```

Required Fields

DDEID, DDEMSG, DDEFLG, DDEADD, DDESOC always required

Appendix D
Functional Descriptions

Functional Descriptions of ALPHADDE.EXE and SOCKS.LIT

The descriptions of ALPHADDE.EXE and SOCKS.LIT that are follow are provided to help you understand how AlphaDDE works. Knowledge of these is not necessary to be able to develop Topic Servers.

AlphaDDE.exe

This is a Microsoft Windows program that functions as a DDE server. AlphaDDE requires a WinSock implementation of TCP/IP via a WinSock interface and protocol stack. The functions of AlphaDDE are:

1. When initiated, AlphaDDE.exe attempts to connect via TCP/IP to an AMOS system running AlphaTCP. The destination system must be running SOCKS (see below) for this connection to be successful. If the connection is not successful, AlphaDDE.exe will retry the connection every few seconds. The address of the destination system is configured in AlphaDDE.INI. Note: AlphaDDE.INI is created by SETUP. To modify it, use an editor such as Notepad.
2. This initiation would be normally be either in a user's STARTUP program group.
3. As part of its initialization, AlphaDDE.exe registers its service (application) for DDE clients to use.
4. When all setup is done, AlphaDDE.exe minimizes to an icon which is the form in which it handles its DDE server function.
5. When a DDE item request from an application arrives, AlphaDDE.exe constructs an ITC-format message in which it passes the service, topic and item strings via TCP/IP to the AMOS host. The ITC message is constructed at this point so as to maintain the same format throughout the connection, and avoid extra copying by allowing address substitution.
   AlphaDDE constructs a Client "return" address from the AlphaNET address in AlphaDDE.INI and the DDE client's handle (used as the task's socket number). This address is put in the message header for SOCKS to use in returning the message.
6. AlphaDDE accepts Topic registration/deregistration messages sent from the AMOS host.
7. AlphaDDE accepts responses for any pending DDE request. These include cold links (a specific request for data was sent), and warm/hot links (data may be returned at any time). At the low level of AlphaDDE processing, there is no difference between warm and hot links.
   When a response is received, it is passed to the application -- either the Item's value, a 'Not Found' status, or some other error status.
8. AlphaDDE supports the System Topic. This provides a Client with access to the current Topics list.
9. AlphaDDE.exe provides a listbox showing the currently available Topics.
10. AlphaDDE provides menu-selectable Trace (debug) information that is output to its own window.
11. If shut down, AlphaDDE must unregister its DDE service (application), send a shutdown message to the host, and close its TCP/IP connection.

SOCKS.LIT

On the AMOS AlphaTCP host, this server program interfaces between TCP/IP and the ITC message system. The functions of SOCKS are:

1. Register a socket with AlphaTCP through the BSD-ish Socket interface (from whence the name SOCKS). This currently uses a well-known-to-this-program socket number of 7777 decimal. This is the target of the TCP/IP connection initiated by AlphaDDE.exe mentioned above.
2. SOCKS is started in the system .INI file, after AlphaTCP.

3. SOCKS registers with the AlphaNET Name Registry using the name DDESOCKS. This allows Topic Servers to find SOCKS. SOCKS only looks for Topic Servers during start-up.
4. Topic Servers register their Topics with SOCKS after finding the address of SOCKS via the Name Registry. The message format is described under messages, above. SOCKS maintains a list of Topics and their associated ITC address to use when it receives messages from AlphaDDE.exe.

   SOCKS waits for messages.
   If one is received from AlphaTCP, it checks to see if it is an AlphaDDE message that is targeted for a known Topic. If so, it updates the destination address to that of the desired Topic Server, and passes the message on to the ITC message system.
   If one is received from ITC, it checks to verify that it is from a Topic Server. If so, it is examined to determine the type. If it is either registering or de-registering a Topic, the Topics list is updated appropriately. If it is data to be returned to a PC, the source address portion of the DDE header is used to return the message to its AlphaDDE.exe on the correct PC.
5. If shut down, SOCKS deregisters its service (in the Name Registry), then sends shutdown messages to AlphaDDE.exe, as well as all registered Topic Servers. Then it closes its TCP/IP and ITC sockets and exits.

Appendix E
AlphaDDE.INI

ALPHADDE.INI is the configuration file for AlphaDDE on the PC. It is initially created by Setup when you install AlphaDDE. If you wish or need to make modifications, you can use an editor such as *Notepad*. By default, it is put in the ALPHADDE directory. If you need to connect via AlphaDDE to different AMOS hosts, you will need to create a copy of ALPHADDE.INI in a separate directory for each host, probably with a Program Group item that refers to each one. ALPHADDE.INI may be put in the WINDOWS directory, if you only need access to a single host.

The only parameter that needs to be changed for different hosts is the [hostaddress] InternetName.

Entries in ALPHADDE.INI

[myaddress]	***This contains address information to be used to identify the PC***
ITCName=	This contains the ITC address used for the PC. This is an AlphaNET-style address used in identifying the PC.

[hostaddress]	***This contains address information about the host***
InternetName=	This contains the Internet (TCP/IP) address of the host running SOCKS. Either a defined system name (AM4000) or a dotted decimal address (ex: 192.0.0.1) is acceptable.
Socket=	7777. Fixed TCP socket number for this release.

[Configuration]	***This contains information to control how AlphaDDE.exe starts***
Minimize=	This controls the initial action of AlphaDDE's window. Options are: 1 - start as normal window 2 - start as icon (minimized). DEFAULT
DisconnectCold=	This option controls whether to stop a hot link when the client program disconnects. Options are: 0 - do not disconnect 1 - disconnect. DEFAULT

[Trace]	***This contains information to control the Trace options at start-up***
Output=	This option sets the output device for Trace information 0 - no Trace output. DEFAULT 1 - Output to Trace window 2 - Output to Debug monitor or window 3 - Output to both windows
Flags=	This option sets which Trace settings are active. The settings are bit flags, so except for the 0 and -1 settings may be OR'd together. DEFAULT setting is 7. The values are: 0 none -1 set all flags 1 General information 2 Errors 4 Topic messages 8 Socks messages 16 DDE messages 32 Dump DDE message data 64 ITC messages 128 Dump ITC message data 256 TCP messages 512 Dump TCP message data 1024 Verbose mode (extra detail)

Example ALPHADDE.INI file:

```
[myaddress]
ITCNAME=W=1,G=1,N=11

[hostaddress]
InternetName=am3000
Socket=7777

[Configuration]
Minimize=2
DisconnectCold=1

[Trace]
Output=1
Flags=7
```

Appendix F
Setting Up AlphaDDE

Setup Configurations:

SSD Number: _____

AlphaDDE Product Installation Code (***pic***): _____-_____-_____-_____

AMOS AlphaDDE account (**DDE:**) (***dev:[ppn]***): dsk0:[7 , 55]

AlphaDDE Installer Information:

AlphaDDE Installation Subdirectory (***ddedir***): c:alphadde

Local Workstation AlphaNET Address (***wrksan***): Network:_____ Group:_____ Node:_____

Remote AMOS host IP Address or Name (***amosnm***): _____._____._____._____

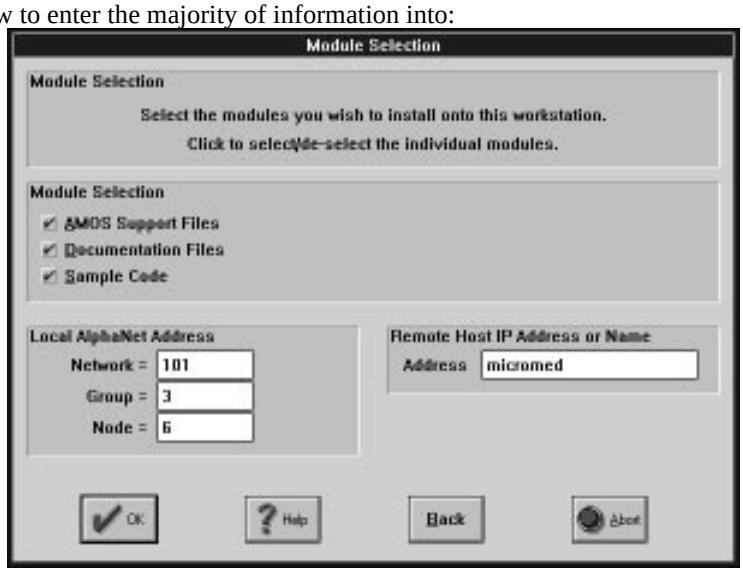
or _____

AlphaDDE Check List

Workstation:

- Install and verify your Windows TCP Stack. The instructions should be provided by the Stack manufacturer (Trumpet installation instructions are provided in this format.)
- If using a name instead of an absolute IP address for ***amosnm*** verify that it is defined in the TCP Stacks local **HOSTS.** file or on the Domain Name Server.
- From the Program Manager of Windows, select **File** menu and choose **Run**. If installing from floppy enter **a:setup.exe** (or **b:\setup.exe**), if installing from the distribution file enter **addeinst.exe** from it's current directory.

- Follow the installation instructions shown. Enter the information gathered above in the correct text fields. You will be requested to enter items ***ddedir***, ***wrksan*** (in it's three parts) and ***amosnm***. You should see a screen like the one below to enter the majority of information into:



- Once installation is complete restart Windows to verify that AlphaDDE starts up.

AMOS:

- Following the *Initial AlphaTCP Setup* instructions, install AlphaTCP and verify its operation.
- Create the account ***dev:[ppn]*** for AlphaDDE.
- Add the ersatz **DDE:=*dev:[ppn]*** to **SYS:TCPIP.ERZ** for the AlphaDDE account.
- Using a PC to AMOS file transfer facility like AlphaLAN or FTP copy the AlphaDDE files into their respective accounts. All the AMOS files needed are copied to the PC during installation if that option was selected. Following shows the PC and AMOS accounts involved:

  C:\ALPHADDE\AMOS\SYS*.* ➔ SYS:*.*
  C:\ALPHADDE\AMOS\BAS*.* ➔ BAS:*.*
  C:\ALPHADDE\AMOS\MAC*.* ➔ MAC:*.*
  C:\ALPHADDE\AMOS\DDE*.* ➔ DDE:*.*

- Execute the **SOCKS** command to enter the Product Installation Code, use ***pic***.
- Copy your main system initialization file to a temporary ini file (TCPIP.INI)
  - Increase the number of **JOBS** by 1.
  - Add the following after the last JOBALC line.
    JOBALC SOCKS
  - Add the following after the last TRMDEF line:
    TRMDEF SOCKS, PSEUDO, NULL, 50, 50, 50
  - Following the TCPEMU job initialization add the following line:
    SETJOB SOCKS, SOCKS, 150K, SOCKS
- Reboot AMOS using the test initialization file and, using STAT, verify **SOCKS** is running in a sleep state (**RN Sl**).

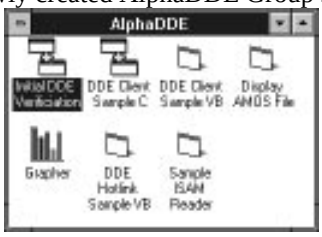
Point to Point Test

AMOS:

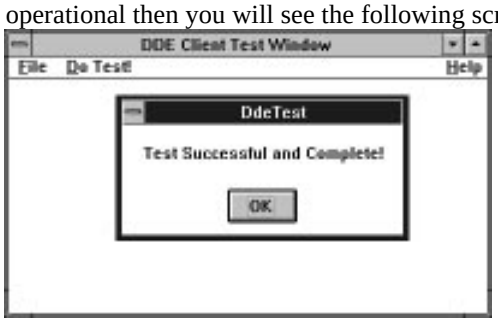
- Log to **DDE:**
- Enter **SMPTOP** from AMOS.

Windows:

- From the newly created AlphaDDE Group select the Initial DDE Verification icon



- Select **Do Test** from the menu bar.
- After a few stats message boxes you should get a screen that indicates the success (or failure) of the test. If everything is operational then you will see the following screen.



- After completion you can ^C the AMOS job.

Index

—(—

(amos) - defined, 13
(assembler) - defined, 13
(basic) - defined, 13
(xcall) - defined, 13

—A—

Advise Link, 5, 35, 38
Advise Reject, 14
Advise request, 7, 16, 24, 36, 49, 50, 53, 67, 68, 72
Advise Start, 7, 10, 14, 38, 50, 52
Advise Stop, 7, 18, 62
ALPHADDE.EXE, 1, 3, 6, 61, 62, 63, 71, 75, 77
AlphaDDE.INI, 75, 77

—C—

client, 5
Cold Link, 5, 61, 62, 75
Cold Request, 20, 22, 25, 29, 31, 33, 47, 71
Ctrl/C, 3, 19, 28, 29, 31, 33, 35, 36, 42, 43, 45, 47
Ctrl-C, 25, 31

—D—

DDE, 1, 5
dde’add’hotlnk, 10, 14, 15, 18, 39, 53
dde’advise’all, 10, 14, 15, 18, 35, 49
dde’connect, 9, 16, 19, 27, 41
dde’delete’hotlnk, 10, 14, 15, 18
dde’disconnect, 9, 17, 19, 28, 42
dde’get’request, 10, 20, 22, 23, 24, 26, 30, 44
dde’process’request, 10, 14, 18, 22, 23, 26, 34, 48
dde’return’string, 10, 24, 37, 51
dde’return’unf, 10, 24, 37, 51
dde’wait’request, 9, 21, 23, 24, 25, 32, 46
DDECON, 9, 17, 19, 22, 27, 32, 33, 40, 42, 45
DDECON(xcall), 9, 27, 28, 41
DDEDIS, 9, 28, 41, 42
DDEDIS(xcall), 9, 17, 27, 28, 42
DDEGET, 10, 16, 21, 24, 26, 30, 32, 33, 34, 36, 37, 40, 43, 45, 46, 47, 50, 51, 52, 57
DDEGET(xcall), 10, 29, 44
DDEGTR, 9, 10, 21, 24, 26, 30, 32, 34, 36, 37, 43, 44, 45, 47, 48, 50, 57
DDEGTR(xcall), 9, 31, 46
DDEML, 5
DDEPRC, 10, 20, 23, 30, 32, 34, 38, 43, 44, 46, 47, 48, 52
DDEPRC(xcall), 10, 33, 48
DDESNA, 10, 15, 24, 35, 36, 38, 39, 49, 50, 52, 53
DDESNA(xcall), 10, 35, 49
DDESNR, 10
DDESNR(xcall), 10, 36, 51
DDESVA, 10, 14, 35, 38, 39, 49, 50, 52
DDESVA(xcall), 10, 38, 53

—E—

Execute, 5, 7, 16, 20, 22, 25, 29, 31, 33, 57, 65, 67, 72

—H—

Hot Link, 5, 10, 14, 16, 18, 20, 22, 24, 25, 29, 30, 31, 32, 33, 35, 36, 38, 43, 45, 47, 49, 50, 52, 61, 62, 66, 67, 72, 75, 77

—I—

item request, 5, 7, 9, 16, 24, 25, 31, 32, 45, 75

—L—

Lotus 123, 1, 57, 59

—M—

Microsoft Access, 60
Microsoft Word, 1, 6, 60

—N—

Name Registry, 3, 6, 16, 20, 22, 25, 28, 29, 31, 33, 40, 41, 42, 67, 69, 76

—O—

OLE, 5

—P—

Poke, 5, 7, 16, 22, 24, 25, 32, 33, 36, 45, 47, 50, 57, 67, 68, 72
Poke Data, 20, 22, 25, 29, 31, 33

—R—

request, 1, 57, 58, 71

—S—

SOCKS.LIT, 1, 3, 4, 5, 6, 61, 62, 75
Start Advise, 38

—T—

TCP/IP, 75
Topic Server, 1, 4, 5, 6, 57, 61, 62, 71, 75, 76

—V—

Visual Basic, 1, 3, 6, 57, 58

—W—

Warm Link, 5, 10, 20, 22, 25, 29, 31, 33, 72